Penjelasan Kode Program router_final

Difficulty-Aware Router untuk YOLOv8n-P2: struktur, alur, dan rincian setiap modul, skrip, uji, dan notebook

2 November 2026

Daftar Isi

Cara Memakai Dokumen Ini 3

1 Gambaran Umum Kode 3

1.1 Apa yang Dikerjakan router_final 3
1.2 Yang Dibuang dari Versi Lama 4
1.3 Peta Folder 4
1.4 Alur Pipeline 5
1.5 Jejak Pemanggilan Utama 5
1.6 Prinsip Desain yang Berulang 6

2 Konfigurasi 6

2.1 Graf Model: yolov8n-dualbranch-p2.yaml 6
2.2 Konfigurasi Eksperimen: kitti.yaml dan visdrone.yaml 7

3 Detektor Dua Cabang: model.py, loss.py, train.py 9

3.1 Membangun DualBranchDetector 9
3.2 Menjalankan Graf per Tahap 10
3.3 Fitur Gate: pool4 10
3.4 Memuat Checkpoint 11
3.5 Loss Dua Cabang 11
3.6 Trainer dan Callback 12

4 Data dan Evaluasi COCO: data.py 12

4.1 DataLoader dan Urutan Citra 12
4.2 Koordinat Asli dan Batas Objek Kecil 13
4.3 NMS dan NMS Cepat Bersertifikat 13
4.4 Ground Truth COCO dan COCOeval 14
4.5 soft_small per Citra 14

5 Gate Post-hoc dan Kendali Anggaran: gate.py, budget.py 15

5.1 Cache Fitur dan Deteksi 15
5.2 Target dan Fit Gate 15
5.3 Melipat Scaler dan Regresi Logistik 16
5.4 Kendali Anggaran B2 16
5.5 Kuantil Jendela Terurut 17

6 Deployment A3: export.py, trt.py, evaluate.py 17

6.1 Antarmuka Engine 17
6.2 Stage 1 dan Presisi 18
6.3 Membangun Operator If 18
6.4 Verifikasi ONNX 19

6.5 Engine TensorRT 19
6.6 Evaluasi Satu Split 20
7 Statistik dan Evaluasi Final: cocomix.py, baselines.py, suite.py 20
7.1 Metrik Routing dari evalImgs: CocoBootstrap, CocoMix, Router 20
7.2 Bootstrap Berpasangan: UnionBoot, FastUnionBoot, boot_engine 21
7.3 Uji, Nilai p, Holm, Korelasi Parsial, Pooling, dan Klaster 22
7.4 Pembanding: Vanilla, P2-Static, Heuristik, Kaskade 22
7.5 Suite Tahap A: Cache, Cek Gate, Ekspor 23
7.6 Job Inferensi TensorRT 24
7.7 Analisis dan Kriteria 24
7.8 Menjalankan, Mengarsip, dan Menggabung 26
8 Analisis Pelengkap: ablation.py, ckcheck.py, profile.py, latency_l2.py, a12.py 26
8.1 Ablasi Pasca-Pelatihan 26
8.2 Cek Checkpoint 27
8.3 Profil Latensi per Segmen 27
8.4 Latensi L2 27
8.5 A12: Engine A3T 28
9 Lingkungan, Provenans, dan Reproduksi: env.py, repro.py, compat.py 29
9.1 Penjaga Ultralytics Stok dan Provenans 29
9.2 Sidik Jari Bobot 29
9.3 Pemetaan Bobot M0 30
10 Skrip Baris Perintah 30
11 Pengujian dan Alat Bantu 32
11.1 Uji Unit dan Uji Kesetaraan 32
11.2 Rantai Validasi terhadap M0 33
11.3 Run Ulang dari Commit Bersih 34
12 Notebook Kaggle 34
13 Keterbatasan dan Catatan tentang Kode 35
14 Pertanyaan Penguji tentang Kode 36
Lampiran A. Padanan Label Kriteria 38
Lampiran B. Indeks Fungsi dan Kelas Penting 39

Cara Memakai Dokumen Ini

Dokumen ini menjelaskan kode program `router_final`, yaitu implementasi sistem final tesis, dari konfigurasi sampai analisis statistik. Tujuannya agar setiap bagian kode dapat dijelaskan dan dipertahankan di sidang: apa fungsinya, mengapa ditulis begitu, dan bagaimana kebenarannya diperiksa. Dokumen ini melengkapi dua dokumen persiapan sidang lain. *Laporan Hasil Eksperimen* (1 November 2026) memuat angka hasil, sedangkan *Contoh Perhitungan Numerik* (1 November 2026) menelusuri satu citra secara numerik. Di sini fokusnya adalah kode: struktur, keputusan desain, dan jebakan yang ditangani.

Versi kode yang dijelaskan adalah commit `84bc2b9ee` pada *branch* `final-arsitektur` (31 Oktober 2026, pesan commit tentang A11 dan A12), dari folder `D:\Tesis\Thesis\Kode Program\ultralytics_folder\router_final`. Salinan di komputer Anda sama dengan commit itu; selisihnya hanya akhir baris (CRLF). Folder berisi 63 berkas: 21 berkas Python di `rfinal/` (20 modul dan `__init__.py`, total 4.083 baris), 17 skrip di `scripts/` (1.956 baris), 13 berkas uji di `tests/` (549 baris), 5 alat di `tools/` (531 baris), 3 konfigurasi, 1 notebook Kaggle, serta `README.md`, `conftest.py`, dan `.gitignore`. Kode berjalan di atas Ultralytics stok 8.3.240 dari PyPI, bukan *fork* `ultralytics/` di root repo.

Potongan kode di dokumen ini disalin apa adanya dari commit tersebut. Bagian yang tidak relevan dipotong dan diberi tanda `# ...`, dan beberapa baris yang terlalu panjang untuk halaman dipenggal tanpa mengubah isinya. Nomor baris tidak dicantumkan karena mudah bergeser. Nama fungsi, berkas, dan kolom keluaran ditulis dengan huruf monospasi. Rujukan "Pers. (n)" mengacu ke persamaan di Subbab III draf paper (`03-metode.md`), yang sama dengan Bab III tesis.

Sebagai pemeriksaan, seluruh uji unit dijalankan ulang untuk dokumen ini di lingkungan Linux CPU (Python 3.13, torch 2.15.0, Ultralytics 8.3.240 stok, onnx 1.23.2, onnxruntime 1.29.0, scikit-learn 1.9.1). Hasilnya 25 uji lolos dan 3 dilewati. Ketiga uji yang dilewati memang membutuhkan berkas acuan dari luar folder ini (keluaran *fork* lama dan *intermediate* Kaggle), sehingga dilewati secara otomatis bila variabel lingkungannya tidak diisi. Lingkungan Kaggle tempat eksperimen berjalan berbeda: Tesla T4, torch 2.11.0+cu128, dan TensorRT. Jadi uji lokal ini membuktikan logika kode, bukan angka latensi.

Penting: dua penomoran kriteria. Kode memakai penomoran protokol evaluasi (H1–H7), sedangkan tesis dan paper memakai penomoran baru (H1–H6). Contohnya, uji berlabel `H6 utama` di `bootstrap_test.csv` adalah kriteria **H5** di tesis. Padanan lengkapnya ada di Lampiran A. Di dokumen ini, label kode ditulis apa adanya dan padanan tesisnya disebut bila perlu.

1 Gambaran Umum Kode

1.1 Apa yang Dikerjakan router_final

Sistem final terdiri atas empat tahap yang dijalankan berurutan, dan kode dibagi mengikuti tahap itu. Tahap pertama melatih detektor dua cabang YOLOv8n-P2 dengan *trunk* bersama (layer 0–15), cabang A dengan jalur P2 (layer 16–27 dan kepala `Detect_A` di layer 34), serta cabang B tanpa P2 (layer 28–33 dan kepala `Detect_B` di layer 35). Kedua cabang dilatih penuh di setiap *batch* dengan kerugian deteksi stok saja, sesuai Pers. (1). Tahap kedua membekukan detektor, lalu melatih gate *post-hoc*: regresi logistik di atas 960 fitur hasil *pooling* empat peta fitur *trunk*, dengan target apakah cabang A menambah deteksi objek kecil pada citra latih [Pers. (2)–(4)]. Tahap ketiga mengubah skor gate menjadi keputusan rute dengan kendali anggaran B2, yaitu ambang kuantil bergulir atas 200 skor sebelumnya [Pers. (5)]. Tahap keempat mengekspor seluruh sistem sebagai satu graf ONNX dengan operator `If`, lalu membangunnya menjadi satu *engine* TensorRT FP16 (disebut A3).

Di atas keempat tahap itu ada lapisan evaluasi. Lapisan ini menghitung metrik COCO pada koordinat asli, menyusun metrik routing apa pun dari hasil per citra tanpa menjalankan COCOeval ulang, menjalankan *bootstrap* berpasangan untuk setiap uji, mengukur latensi TensorRT, dan menilai kriteria yang dikunci. Analisis pelengkap (ablasi, cek *checkpoint*, profil latensi, L2, sapuan ρ, *bootstrap* klaster, A11, dan A12) memakai modul yang sama.

Sebagian besar kode evaluasi adalah *port* dari skrip eksperimen versi M0 di folder `eksperimen/`: `tahap2_eval.py`, `tahap2b_plafon.py`, `tahap3a.py`, `tahap4_final.py`, dan `tahap4b.py`. Bagian yang menyentuh model diganti dengan modul baru, sedangkan analisis statistiknya disalin tanpa perubahan semantik. Konsekuensinya, kesetaraan dengan kode lama dapat dibuktikan dengan uji, dan memang dibuktikan (Bab 11).

1.2 Yang Dibuang dari Versi Lama

Versi pertama (M0) memasang modul keputusan di dalam graf detektor. Tabel 1.1 merangkum komponen yang dibuang dan alasannya menurut README. Semua komponen itu hanya melayani gate terlatih versi lama. Dengan gate *post-hoc*, kendali aktivasi dipindah ke luar pelatihan.

Tabel 1.1. Komponen versi lama yang tidak ada di `router_final`.

Dibuang	**Alasan**
`UltraLightWeightDifficultyAwareRouter` (`squeeze_p3/p2`, `classifier`, `tau_param`, Gumbel-softmax)	Gate terlatih tidak dipakai sistem final; routing dilakukan gate *post-hoc*.
Cabang *proxy* (`proxy_stem/cls/reg_dist`) dan `compute_proxy_supervision_loss`	Hanya memodulasi *sparsity penalty*.
`gate_value_loss` (BCE, Opsi C dengan *offset* EMA) dan `compute_router_loss` (*sparsity*, `difficulty_weight`)	Kendali aktivasi dipindah ke pasca-pelatihan (B2).
Kalibrasi `calib_T/calib_b`, `compute_gate_only`, tiga *engine* terpisah	Diganti skor dan ambang B2 di *engine* A3.
`gate_signal_mode`, `normalize_bg_anchors`, loss per strata ukuran, `router_buffer`	Hanya melayani sinyal gate lama.

Pembuangan ini tidak mengubah jaringan deteksi. Bobot M0 yang dipetakan ke graf baru memberi fitur *trunk* dan keluaran kedua cabang yang identik bit per bit dengan model lama (Subbab 9.3 dan 11.1). Yang berubah adalah resep pelatihan, karena loss router tidak lagi ikut ke *gradient clipping* dan GradScaler. Karena itu bobot final berbeda dari M0, dan semua angka dihasilkan ulang dari model final.

1.3 Peta Folder

Tabel 1.2 memetakan isi folder. Modul di `rfinal/` adalah pustaka, sedangkan skrip di `scripts/` adalah antarmuka baris perintah yang memanggil pustaka itu. Notebook Kaggle hanya menjalankan skrip secara berurutan.

Tabel 1.2. Isi folder `router_final` (jumlah baris pada commit `84bc2b9`).

Folder	**Berkas (baris)**	**Peran**
`configs/`	`yolov8n-dualbranch-p2.yaml` (66), `kitti.yaml` (79), `visdrone.yaml` (87)	Graf model dan seluruh angka eksperimen yang dikunci
`rfinal/` inti	`model.py` (203), `loss.py` (40), `train.py` (146), `data.py` (186), `gate.py` (110), `budget.py` (111)	Detektor, loss, pelatihan, data dan COCO, gate, kendali anggaran
`rfinal/` deployment	`export.py` (180), `trt.py` (71), `evaluate.py` (99)	ONNX `If`, TensorRT, evaluasi satu split
`rfinal/` evaluasi final	`suite.py` (1.180), `cocomix.py` (417), `baselines.py` (116)	Protokol H1–H7, statistik, pembanding

Folder	Berkas (baris)	Peran
`rfinal/` analisis pelengkap	`ablation.py` (262), `ckcheck.py` (108), `profile.py` (221), `latency_l2.py` (228), `a12.py` (196)	Ablasi, cek *checkpoint*, profil latensi, L2, A3T
`rfinal/` pendukung	`env.py` (132), `compat.py` (43), `repro.py` (32)	Penjaga lingkungan dan provenans, pemetaan M0, sidik jari bobot
`scripts/`	16 skrip CLI dan `_path.py`	Satu skrip per langkah pipeline (Bab 10)
`tests/`	13 berkas `test_*.py`	Uji kesetaraan dan uji unit (Bab 11)
`tools/`	`m0_state_dict.py`, `m0_reference.py`, `convert_m0.py`, `compare_suite_runs.py`, `b8_sweep_rho_bersih.py`	Rantai validasi terhadap M0 dan run ulang dari commit bersih
`notebooks/`	`kaggle_router_final.ipynb` (18 sel)	Urutan eksekusi di Kaggle T4 per sesi (Bab 12)

1.4 Alur Pipeline

Tabel 1.3 menunjukkan urutan eksekusi untuk satu *seed*, beserta modul dan keluaran utamanya. Setiap langkah membaca keluaran langkah sebelumnya dari disk, sehingga satu langkah dapat diulang tanpa mengulang langkah lain. Pola ini juga yang memungkinkan VisDrone dipecah menjadi tiga sesi Kaggle.

Tabel 1.3. Urutan pipeline per *seed*.

No	Langkah	Skrip	Modul utama	Keluaran utama
0	Uji unit	`pytest tests`	semua	lolos/gagal
1	Uji reproduksi GPU	`check_reproducibility.py`	`train`, `repro`	`state_hash.json` dua run
2	Pelatihan detektor	`train.py`	`model`, `loss`, `train`	`weights/last.pt`, `results.csv`, `branch_b_metrics.csv`, `provenance.json`
3	Pelatihan pembanding	`train.py --model vanilla/p2static`	`train`	`last.pt` Vanilla dan P2-Static
4	Cek *checkpoint*	`check_checkpoint.py`	`ckcheck`	`titik.csv`, `uji.csv`, `plafon_oracle.csv`
5	Gate	`fit_gate.py`	`gate`, `data`, `budget`	`gate.json` (w, b, thr0)
6	Ekspor A3	`export.py --trt`	`export`, `trt`	`a3_fp16.onnx`, `a3_fp16.engine`, `deploy.json`
7	Evaluasi cepat	`evaluate.py`	`evaluate`	`metrik.csv`, `keputusan_per_gambar.csv`
8	Cek NMS cepat (VisDrone)	`check_fast_nms.py`	`data`	`ringkasan.csv`
9	Evaluasi final terkunci	`suite.py run`	`suite`, `cocomix`, `baselines`	`hasil.json`, `bootstrap_test.csv`, `coco_lengkap.csv`, `latensi*.csv`
10	Ablasi	`ablate.py`	`ablation`	`ablasi.csv`, `ablasi_uji.csv`
11	Gabungan	`suite.py merge`	`suite`	`ringkasan_*.csv`, `pareto.png`
12	Pasca-hasil	`profile_latency.py`, `h6_latency.py`, `sweep_rho.py`, `cluster_boot.py`, `a11_*`, `a12_*`	`profile`, `latency_l2`, `a12`	`segmen.csv`, `l2.json`, dan lain-lain

1.5 Jejak Pemanggilan Utama

Saat sidang, pertanyaan tentang kode biasanya berbentuk “bagian mana yang menjalankan X”. Jejak pemanggilan berikut menjawab pertanyaan itu untuk empat jalur utama.

Pelatihan. `scripts/train.py` memanggil `build_trainer()` di `rfinal/train.py`, yang membuat `DualBranchTrainer` (turunan `DetectionTrainer` stok). Ultralytics memanggil `get_model()`, yang

membangun `DualBranchDetector` dari YAML. Pada setiap *batch*, trainer stok memanggil `model(batch)`. Karena masukannya *dict*, `BaseModel.forward` meneruskannya ke `DualBranchDetector.loss()`, yang menjalankan `forward_dual()` lalu `DualBranchLoss.__call__()`. Fungsi itu memanggil dua `BranchLoss` (`v8DetectionLoss` stok), satu per kepala.

Gate. `scripts/fit_gate.py` memanggil `cache_split()` (satu *forward* per citra: `trunk()`, `pool4()`, `BranchA`, `BranchB`, `to_dets()`), lalu `soft_small_delta()` (COCOeval kedua cabang, lalu `per_image_soft_small()`), lalu `fit_gate()`, dan akhirnya `thr0_from_val()` pada skor val.

Inferensi yang di-*deploy*. Host memanggil `B2Controller.threshold()`, menjalankan *engine* A3 dengan citra, `mode = −1`, dan ambang. Di dalam *engine*, Stage 1 menghitung *trunk* dan skor gate, operator `If` memilih Stage 2A atau 2B, dan keluarannya `y`, `use_a`, `score`. Host lalu memanggil `B2Controller.push(score)` dan menjalankan NMS (`to_dets()`). Di kode, jalur ini ada di `A3Engine.infer()` (`trt.py`), `run_split()` (`evaluate.py`), dan `e2e_pass()` (`suite.py`).

Evaluasi final. `scripts/suite.py run` memanggil `run_model()`. Fungsi ini menjalankan `stage_a()` (PyTorch: *cache*, cek gate, ekspor, verifikasi), lalu `job_main()` di proses terpisah (TensorRT: pass akurasi, latensi, ujung-ke-ujung), lalu `analyze()` (CocoMix, *bootstrap*, kriteria).

1.6 Prinsip Desain yang Berulang

Beberapa prinsip muncul di hampir semua modul, dan memahaminya mempermudah membaca kode. Prinsip pertama adalah **satu sumber kebenaran**. Semua angka eksperimen ditulis di `configs/*.yaml`, dan semua skrip membacanya; sha256 berkas konfigurasi tercatat di `provenance.json` setiap run. Bobot gate hanya disimpan di `gate.json`, dan suite tidak memakai hasil fit ulangnya sendiri, tetapi memakai fit ulang itu sebagai pemeriksaan silang.

Prinsip kedua adalah **gagal cepat**. Kode penuh dengan `assert` yang menghentikan eksekusi bila asumsi dilanggar: jumlah citra split tidak sesuai `expected_counts`, layer `Detect` tidak di indeks yang diharapkan, `gate.json` berbeda dari fit ulang, keputusan *engine* berbeda dari perbandingan di host, atau metrik CocoMix berbeda dari COCOeval langsung. Kegagalan seperti ini lebih murah daripada angka yang salah tanpa disadari.

Prinsip ketiga adalah **urutan per citra dijaga**. Hampir semua analisis berupa array per citra (skor, keputusan, deteksi) yang harus sejajar dengan daftar citra. Karena itu semua *loader* evaluasi memakai *batch* 1, `shuffle=False`, dan `DataLoader` biasa (Subbab 4.1), dan ada konversi eksplisit antara urutan *loader* dan urutan `imgIds` COCO (`Router.sorted_mask()`).

Prinsip keempat adalah **kesetaraan dibuktikan, bukan diklaim**. Setiap *port* atau optimasi yang seharusnya tidak mengubah hasil disertai uji: B2 efisien bit-identik dengan `np.quantile`, NMS cepat identik dengan NMS penuh atau kembali ke NMS penuh, `UnionBoot` identik dengan COCOeval, dan ONNX identik dengan PyTorch dalam toleransi. Prinsip kelima adalah **provenans**: setiap run menulis commit, status *dirty*, versi pustaka, GPU, *driver*, dan sha256 masukan, sehingga setiap angka dapat dilacak ke kode yang menghasilkannya.

2 Konfigurasi

2.1 Graf Model: yolov8n-dualbranch-p2.yaml

Graf detektor ditulis dalam format YAML Ultralytics dan hanya memakai modul stok: `Conv`, `C2f`, `SPPF`, `Concat`, `nn.Upsample`, dan `Detect`. Tidak ada modul router, gate terlatih, cabang *proxy*, atau kalibrasi di dalam graf. Keputusan routing dibuat di luar graf, dari fitur yang sudah dihasilkan *trunk*.

Skala lebar `n: [0.33, 0.25, 1024]` membuat jumlah kanal tertulis dikali 0,25 dan jumlah blok dikali 0,33, sama dengan YOLOv8n.

Tabel 2.1 menunjukkan setiap layer dengan bentuk keluaran pada masukan 640 × 640 dan jumlah parameternya. Angka ini diukur dari model yang dibangun dari YAML tersebut, sebelum *fuse* Conv–BN, sehingga parameter BatchNorm ikut terhitung.

Tabel 2.1. Layer graf dua cabang (masukan 640 × 640).

Layer	Modul	Masukan dari	Keluaran	Parameter	Peran
0–1	Conv, Conv	−1	16×320², 32×160²	5.136	*stem*
2	C2f	−1	32×160×160	7.360	**`p2_backbone`** (fitur gate dan masukan jalur P2)
3–8	Conv/C2f	−1	s.d. 256×20×20	1.095.552	*backbone* P3–P5
9	SPPF	−1	256×20×20	164.608	**`p5_sppf`**
10–12	Upsample, Concat(6), C2f	−1	128×40×40	148.224	**`p4_neck`**
13–15	Upsample, Concat(4), C2f	−1	64×80×80	37.248	**`p3_neck`** (juga skala P3 cabang B)
16–18	Upsample, Concat(2), C2f	−1	32×160×160	9.408	jalur P2 cabang A (“expert P2”)
19–27	Conv/Concat/C2f	18, 15, 12, 9	64×80², 128×40², 256×20²	1.078.784	*bottom-up* cabang A
28–33	Conv/Concat/C2f	15, 12, 9	128×40², 256×20²	801.408	*bottom-up* cabang B
34	Detect	18, 21, 24, 27	6 × 34.000	617.496	`Detect_A`, *stride* 4, 8, 16, 32
35	Detect	15, 30, 33	6 × 8.400	751.702	`Detect_B`, *stride* 8, 16, 32

Trunk (layer 0–15) memiliki 1.458.128 parameter dan total model 4.716.926. Jumlah *anchor* 34.000 berasal dari $160^2 + 80^2 + 40^2 + 20^2$ dan 8.400 dari $80^2 + 40^2 + 20^2$. Dengan penghitung FLOPs PyTorch yang juga dipakai suite (`FlopCounterMode`, dua FLOP per perkalian-tambah), *trunk* butuh 4,10 GFLOPs, cabang A 8,51 GFLOPs, dan cabang B 3,98 GFLOPs. Jadi rute A sekitar 12,61 GFLOPs dan rute B sekitar 8,08 GFLOPs; gate sendiri praktis nol.

Ada satu hal yang sering mengejutkan: `Detect_A` punya parameter lebih sedikit daripada `Detect_B`, padahal kepalanya empat skala. Penyebabnya adalah aturan lebar di `Detect` stok. Lebar cabang klasifikasi ditetapkan sekali dari kanal skala pertama, yaitu `c3 = max(ch[0], min(nc, 100))`. Skala pertama `Detect_A` adalah P2 dengan 32 kanal, sehingga `c3 = 32` untuk keempat skalanya. Skala pertama `Detect_B` adalah P3 dengan 64 kanal, sehingga `c3 = 64`. Cabang regresi kotak sama lebarnya (`c2 = 64`) di kedua kepala.

Layer 16–18 dibuat identik secara numerik dengan `compute_expert` versi lama, yaitu `C2f(96 → 32, n = 1, shortcut = False)` setelah *upsample* dan *concat*. Kesamaan inilah yang memungkinkan bobot M0 dipetakan ke graf baru untuk uji kesetaraan. Satu perbedaan struktural dengan P2-Static stok (`yolov8n-p2.yaml`) ada di layer 19: konvolusi *downsampling* P2 → P3 di cabang A ditulis `Conv [256, 3, 2]` (64 kanal), sedangkan di P2-Static `Conv [128, 3, 2]` (32 kanal).

2.2 Konfigurasi Eksperimen: kitti.yaml dan visdrone.yaml

Berkas `kitti.yaml` adalah satu-satunya tempat angka eksperimen KITTI ditulis, dan `visdrone.yaml` adalah turunannya. Komentar di kepala berkas menyatakan prinsip ini, dan `provenance.json` mencatat sha256 berkas yang dipakai. Isinya dibagi menjadi blok yang mengikuti tahap pipeline. Tabel 2.2 merangkum nilai kuncinya.

Tabel 2.2. Nilai konfigurasi yang dikunci.

Blok	Kunci	KITTI	VisDrone	Dipakai oleh
`train`	epochs, imgsz, optimizer, lr0, cos_lr, weight_decay	100, 640, AdamW, 0,002, ya, 0,0005	sama	`train.py`
`train`	batch	16	8	`train.py`
`train`	amp, workers, deterministic,	ya, 4, ya, 10	sama	`train.py`

Blok	Kunci	KITTI	VisDrone	Dipakai oleh
	save_period			
seeds		42, 7, 2026	42	notebook
loss	branch_b_weight	0,7	0,7	DualBranchLoss
monitor	branch_b_val_every	10	10	_cb_branch_b
checkpoint		last.pt	last.pt	semua tahap
gate	fit_split, target, C, max_iter	train, soft_small, 0,001, 5000	sama	fit_gate.py
gate.nms	conf, iou, max_det	0,001, 0,7, 300	sama, + fast_nms_k 5000	cache_split
budget	rho, window, min_hist	0,5, 200, 20	sama	B2
eval	coco_max_dets	1, 10, 100	1, 10, 500	coco_eval
deploy	opset, precision	17, fp16	sama	export.py
analysis	max_det, n_boot, boot_seed	300, 1000, 20261016	500, 1000, 20261016	suite.py
analysis	n_draws, n_draws_sekunder, h5_ms, h7_dev	5, 3, 0,15, 0,05	sama	analyze
analysis	n_builds, n_prof_images, n_warmup, n_reps	2, 300, 30, 3	sama	job_main
analysis	cluster_by, cluster_seed	—	video_prefix, 20261017	*bootstrap* klaster

Beberapa pilihan di tabel ini perlu dijelaskan karena tidak tampak jelas dari namanya. Jumlah *worker* (`workers: 4`) termasuk resep, karena menentukan aliran augmentasi acak; mengubahnya mengubah bobot hasil pelatihan meskipun *seed* sama. Kunci `checkpoint: last.pt` dikunci sebelum pelatihan karena `best.pt` Ultralytics dipilih dari *fitness* cabang A saja (Subbab 8.2). VisDrone memakai `coco_max_dets` [1, 10, 500] karena rata-rata sekitar 59 objek per citra dan sebagian citra punya lebih dari 100 objek; dengan batas 100, *recall* tertinggi terpotong. Analisis VisDrone memakai NMS `max_det` 500, sedangkan gate tetap di-fit dengan 300 deteksi teratas (`gate.nms`), dan suite menjaga kesamaan ini (Subbab 7.5).

Nilai `boot_seed` sama untuk semua model. Artinya, indeks *resample bootstrap* sama untuk semua model yang berbagi data uji, sehingga selisih tiga *seed* KITTI dapat dirata-rata per *resample* (*pooling*, Subbab 7.3). Untuk VisDrone, `cluster_by: video_prefix` menambahkan *bootstrap* klaster per video sumber, dengan ID video diambil dari token pertama nama berkas.

Ada dua catatan kecil yang perlu diketahui. Di `visdrone.yaml`, komentar `# TODO` masih tertulis di samping `data` dan `expected_counts`, padahal nilainya (548 citra val dan 1.610 citra uji) sudah dipakai dan diperiksa oleh `make_loader` di setiap run. Komentar itu sisa penulisan awal, bukan pekerjaan yang belum selesai. Selain itu, `fast_nms_k` ditambahkan setelah sesi 1 VisDrone; `gate.json` dan `eval_test_trt` sesi itu dihitung dengan NMS penuh, dan kesamaan NMS cepat diverifikasi sebelum suite (Subbab 4.3).

3 Detektor Dua Cabang: model.py, loss.py, train.py

3.1 Membangun DualBranchDetector

`DualBranchDetector` adalah turunan `DetectionModel` stok. Konstruktor induk membangun semua layer dari YAML dan mengukur *stride*, tetapi hanya untuk layer terakhir (`model[-1]`), yaitu `Detect_B`. Karena itu konstruktor turunan mengukur *stride* `Detect_A` dengan cara yang sama seperti Ultralytics: satu *forward* pada citra kosong 256 × 256 dengan kepala `Detect` dalam mode *training*, sehingga keluarannya berupa peta fitur mentah yang ukurannya dapat dibagi.

```
def __init__(self, cfg=DEFAULT_YAML, ch=3, nc=None, verbose=True):
    super().__init__(cfg=str(cfg) if isinstance(cfg, Path) else cfg, ch=ch, nc=nc, verbose=verbose)
    self._check_layout()
    self._init_done = True
    s = 256
    self.model.eval()
    self.detect_A.training = self.detect_B.training = True
    with torch.no_grad():
        out_a, out_b = self.forward_dual(torch.zeros(1, ch, s, s))
    self.detect_A.stride = torch.tensor([s / x.shape[-2] for x in out_a])
    assert torch.equal(self.detect_B.stride, torch.tensor([s / x.shape[-2] for x in out_b]))
    self.detect_A.bias_init()     # Detect_B sudah di super().__init__()
    self.stride = self.detect_A.stride
    for d in (self.detect_A, self.detect_B):
        d.shape = None            # buang anchors cache dari forward dummy
    self.model.train()
    assert self.detect_A.stride.tolist() == [4.0, 8.0, 16.0, 32.0], self.detect_A.stride
    assert self.detect_B.stride.tolist() == [8.0, 16.0, 32.0], self.detect_B.stride
```

Ada empat hal yang perlu diperhatikan di sini. Pertama, `bias_init()` dipanggil untuk `Detect_A`, karena konstruktor induk hanya menginisialisasi bias kepala terakhir; tanpa ini bias klasifikasi `Detect_A` tidak mengikuti inisialisasi prior YOLOv8. Kedua, `self.stride` model diisi *stride* `Detect_A` (maksimum 32, sama dengan `Detect_B`), karena Ultralytics memakainya untuk ukuran *grid* dataset dan validator. Ketiga, `d.shape = None` membuang *cache anchor* hasil *forward* 256 piksel, supaya *anchor* dihitung ulang pada ukuran masukan yang sebenarnya. Keempat, penanda `_init_done` membedakan dua fase: selama konstruktor induk berjalan, `_predict_once` memakai jalur bawaan (keluaran layer terakhir) agar induk dapat mengukur *stride* `Detect_B`; setelah itu, jalur khusus dua cabang yang dipakai.

Dua kepala diakses lewat *property*, bukan atribut. Alasannya tertulis di komentar kode: menyimpan modul sebagai atribut membuat PyTorch mendaftarkannya dua kali di `state_dict`, sehingga muncul kunci alias `detect_A.*` di samping `model.34.*`. Model lama memang punya alias itu, dan `compat.py` harus membuangnya saat memetakan bobot M0.

```
@property
def detect_A(self):
    return self.model[L_DET_A]

def _apply(self, fn):
    self = super()._apply(fn)
    if getattr(self, "_init_done", False):
        m = self.model[L_DET_A]
        m.stride, m.anchors, m.strides = fn(m.stride), fn(m.anchors), fn(m.strides)
    return self
```

Metode `_apply` adalah tempat PyTorch menerapkan `.to(device)`, `.half()`, dan sejenisnya. `BaseModel._apply` Ultralytics hanya memindahkan `stride`, `anchors`, dan `strides` milik `model[-1]`. Tanpa *override* ini, tensor `Detect_A` tetap di CPU atau tetap FP32 setelah model dipindah ke GPU, misalnya saat `final_eval` Ultralytics memuat `best.pt`. Terakhir, `_check_layout()` memastikan YAML sesuai asumsi kode: `Detect` hanya di layer 34 dan 35, masukan `Detect_A` adalah layer 18, 21, 24, 27, masukan `Detect_B` adalah 15, 30, 33, dan cabang B mulai dari `p3_neck` (layer 15).

3.2 Menjalankan Graf per Tahap

Ultralytics menjalankan graf dengan `_predict_once`, yang menyimpan keluaran setiap layer di daftar `y` agar layer `Concat` dapat mengambil keluaran layer sebelumnya. Kode final menulis ulang mekanisme itu dalam fungsi kecil `_run`, dengan `y` berupa *dict* indeks layer, sehingga graf dapat dipotong menjadi tahap.

```
def _run(layers, x, y):
    for m in layers:
        if m.f != -1:
            x = y[m.f] if isinstance(m.f, int) else [x if j == -1 else y[j] for j in m.f]
        x = m(x)
        y[m.i] = x
    return x

def trunk(self, x):
    y = {}
    p3 = _run(self.model[TRUNK.start:TRUNK.stop], x, y)
    return p3, y[L_P2_BACKBONE], y[L_P4], y[L_P5]

def branch_a(self, p3, p2, p4, p5):
    y = {L_P3: p3, L_P2_BACKBONE: p2, L_P4: p4, L_P5: p5}
    _run(self.model[BRANCH_A.start:BRANCH_A.stop], p3, y)
    return _run([self.model[L_DET_A]], None, y)
```

`trunk()` mengembalikan empat tensor dalam urutan tetap (`p3_neck`, `p2_backbone`, `p4_neck`, `p5_sppf`), dan urutan ini dipakai di mana-mana, termasuk sebagai nama tensor ONNX (`FEATS_A`). `branch_a()` dan `branch_b()` mengisi *dict* `y` hanya dengan keluaran *trunk* yang dibutuhkan, lalu menjalankan layer cabangnya. Bila sebuah layer cabang membutuhkan keluaran yang tidak ada di *dict*, kode akan gagal dengan `KeyError`; jadi pemotongan graf ini sekaligus memeriksa bahwa cabang benar-benar hanya bergantung pada *trunk*. Uji `test_stages_equal_full_forward` memastikan `trunk` diikuti `BranchA` atau `BranchB` identik (`torch.equal`) dengan *forward* penuh.

Jalur bawaan Ultralytics (`_predict_once`, dipakai `val` dan `predict`) mengembalikan cabang A, kecuali atribut `eval_branch` bernilai `"B"`. Atribut itu hanya dipasang oleh `BranchBValidator` selama validasi cabang B. Kelas pembungkus `Trunk`, `BranchA`, dan `BranchB` mengubah fungsi-fungsi tadi menjadi `nn.Module` yang dapat diekspor ke ONNX. Dalam mode *eval*, `Detect` mengembalikan tuple `(y, peta fitur)`; pembungkus mengambil `y` saja, yaitu tensor (1, 4 + nc, N) berisi kotak xywh dalam piksel *letterbox* yang sudah di-*decode* (DFL dan *anchor*) serta skor kelas sigmoid.

3.3 Fitur Gate: pool4

Fitur gate dibuat oleh satu fungsi pendek yang dipakai bersama oleh pelatihan gate, evaluasi, dan *engine* [Pers. (2)].

```
def pool4(p3, p2, p4, p5):
    pool = lambda t: torch.cat([t.mean((2, 3)), t.amax((2, 3))], 1)
    return torch.cat([pool(p3), pool(p2), pool(p4), pool(p5)], 1).float()
```

Untuk setiap peta, `mean((2, 3))` adalah GAP dan `amax((2, 3))` adalah GMP per kanal. Urutan vektor 960 dimensi adalah GAP `p3` (64), GMP `p3` (64), GAP `p2` (32), GMP `p2` (32), GAP `p4` (128), GMP `p4` (128), GAP `p5` (256), dan GMP `p5` (256). Urutan ini harus konsisten dengan `feature_columns()` di `ablation.py`, yang memilih subset kolom untuk ablasi fitur. Panggilan `.float()` di akhir memastikan fitur selalu FP32, termasuk ketika *trunk* berjalan FP16 di *engine*.

3.4 Memuat Checkpoint

`load_detector()` adalah satu-satunya pemuat detektor di seluruh kode analisis, sehingga semua tahap memakai bobot yang sama.

```
ck = torch.load(path, map_location="cpu", weights_only=False)
key = key or ("ema" if ck.get("ema") is not None else "model")
src = ck[key]
m = DualBranchDetector(cfg=src.yaml, nc=int(src.yaml["nc"]), verbose=False)
m.load_state_dict({k: (v.float() if v.is_floating_point() else v) for k, v in
src.state_dict().items()},
                  strict=True)
```

Aturan pemilihan kunci mengikuti Ultralytics. Pada akhir pelatihan, Ultralytics menjalankan `strip_optimizer` pada `last.pt` dan `best.pt`: bobot EMA disalin ke kunci `model`, disimpan dalam FP16, lalu kunci `ema` dikosongkan. Jadi untuk `last.pt` final, kunci yang dipakai adalah `model`, dan isinya adalah bobot EMA. Untuk *checkpoint* periodik (`epochN.pt`), kunci `ema` masih ada dan dipakai. Bobot FP16 di berkas diubah ke FP32 saat dimuat. `strict=True` menjamin semua kunci cocok; bila ada satu kunci yang hilang atau berlebih, pemuatan gagal. Model dikembalikan dalam mode *eval* dengan gradien dimatikan.

Fungsi `reset_anchor_cache()` membuang *cache anchor* setiap kepala `Detect`. Fungsi ini dipanggil setelah `deepcopy`, `.to()`, atau `.half()`, karena `Detect` menyimpan *anchor* sebagai atribut biasa yang tidak ikut dipindah oleh PyTorch.

3.5 Loss Dua Cabang

`loss.py` hanya 40 baris karena seluruh perhitungan diserahkan ke `v8DetectionLoss` stok. Masalah yang diselesaikan hanya satu: `v8DetectionLoss` selalu membaca konfigurasi dari `model.model[-1]`, yaitu `Detect_B`. `BranchLoss` menimpa `stride` dengan *stride* kepala yang dimaksud, setelah memastikan jumlah kelas dan `reg_max` kedua kepala sama (atribut lain seperti `no`, `proj`, dan *assigner* hanya bergantung pada kedua nilai itu).

```
class BranchLoss(v8DetectionLoss):
    def __init__(self, model, head, tal_topk: int = 10):
        super().__init__(model, tal_topk=tal_topk)
        assert head.nc == self.nc and head.reg_max == self.reg_max, \
            "kedua head harus punya nc & reg_max sama"
        self.stride = head.stride

class DualBranchLoss:
    names = ("box_A", "cls_A", "dfl_A", "box_B", "cls_B", "dfl_B")
    def __call__(self, preds, batch):
        out_A, out_B = preds
        la, items_a = self.loss_A(out_A, batch)      # la = vektor 3 (sudah × batch_size)
        lb, items_b = self.loss_B(out_B, batch)
        total = la.sum() + self.w_B * lb.sum()
        return total, torch.cat([items_a, items_b])
```

Setiap `BranchLoss` punya `TaskAlignedAssigner` sendiri (topk 10, $\alpha = 0{,}5$, $\beta = 6$), sehingga penugasan *anchor* positif kedua cabang independen. `v8DetectionLoss` mengembalikan vektor tiga komponen (kotak × 7,5, klasifikasi × 0,5, DFL × 1,5) yang sudah dikali ukuran *batch*, sehingga `total` sesuai Pers. (1) dengan $\lambda_B = 0{,}7$. Enam item terpisah dikembalikan untuk log `results.csv`. Bobot λ_B dibaca dari atribut `branch_b_loss_weight` yang dipasang trainer dari konfigurasi.

Di sisi model, `DualBranchDetector.loss()` mengabaikan argumen `preds` dan selalu menjalankan `forward_dual()` ulang. Alasannya, validator Ultralytics dan mode `compile` hanya memberi keluaran jalur bawaan (cabang A), padahal loss membutuhkan kedua cabang. Uji `test_loss_six_items_and_backward` memeriksa bahwa loss berupa skalar dengan enam item dan bahwa **semua** parameter menerima gradien. Dengan kata lain, tidak ada modul mati di graf.

3.6 Trainer dan Callback

`DualBranchTrainer` hanya menimpa dua metode `DetectionTrainer`. `get_model()` membangun `DualBranchDetector` dan memasang λ_B, sedangkan `get_validator()` mengganti nama loss menjadi enam item. Validasi per *epoch* dan pemilihan `best.pt` tetap perilaku bawaan, yaitu cabang A. Selebihnya (optimizer, jadwal LR, augmentasi, EMA, AMP, *checkpoint*) adalah kode stok.

Tiga *callback* ditambahkan. `_cb_branch_b` berjalan di akhir setiap *epoch*, tetapi hanya bekerja setiap `branch_b_val_every` *epoch* dan di *epoch* terakhir. Ia membuat `BranchBValidator` (validator stok yang memasang `eval_branch = "B"` selama validasi dan selalu memulihkannya di blok `finally`), lalu menulis mAP cabang B ke `branch_b_metrics.csv`. Penjaga `_branch_b_logged_epoch` mencegah baris ganda, karena `final_eval` memanggil *callback* yang sama sekali lagi. Pemantauan ini tidak mengubah pelatihan.

`_cb_provenance` berjalan setelah rutin pra-pelatihan. *Callback* ini menghentikan pelatihan bila AMP efektif berbeda dari yang diminta. Pemeriksaan `check_amp` Ultralytics dapat diam-diam mematikan AMP bila uji AMP-nya gagal atau perangkat sedang luring, dan itu berarti resep berubah tanpa terlihat. Setelah itu, *callback* ini menulis `provenance.json`. `_cb_final_hash` berjalan di akhir pelatihan dan menulis `weights_sha256.json` (sha256 berkas) serta `state_hash.json` (sidik jari tensor, Subbab 9.2).

`build_trainer()` menggabungkan blok `train` konfigurasi dengan *seed*, nama run, dan *override*, lalu memanggil `require_stock_ultralytics()` sebelum apa pun dibuat. Opsi `strict_determinism` memasang `torch.use_deterministic_algorithms(True)` tanpa `warn_only`, sehingga operasi non-deterministik langsung menimbulkan galat; opsi ini untuk diagnosis bila uji reproduksi gagal. `build_baseline_trainer()` melatih Vanilla (`yolov8n.yaml`) dan P2-Static (`yolov8n-p2.yaml`) stok dari nol dengan blok `train` yang sama, sehingga resep dan GPU pembanding setara dengan detektor dua cabang.

4 Data dan Evaluasi COCO: data.py

4.1 DataLoader dan Urutan Citra

Semua evaluasi memakai format yang sama dengan *deployment*: *letterbox* persegi 640, *batch* 1, tanpa augmentasi, dan urutan berkas tetap. `make_loader()` membuat `YOLODataset` stok dengan `augment=False` dan `rect=False`, lalu membungkusnya dengan `DataLoader` biasa milik PyTorch.

```python
def make_loader(data, split, batch=1, imgsz=640, workers=2):
    """
    DataLoader biasa (BUKAN InfiniteDataLoader Ultralytics: iterator persistennya menggeser urutan
    bila loop di-`break`, sehingga hasil per gambar tidak sejajar lagi dengan dataset.labels).
    """
    ds = YOLODataset(img_path=data[split], data=data, task="detect", imgsz=imgsz, augment=False,
                     rect=False, cache=False, batch_size=batch)
    exp = data.get("_expected_counts", {}).get(split)
    assert exp is None or len(ds) == exp, f"split {split}: {len(ds)} gambar, diharapkan {exp}"
    return DataLoader(ds, batch_size=batch, shuffle=False, num_workers=workers,
                      collate_fn=ds.collate_fn, ...)
```

Pilihan `DataLoader` biasa ini penting. `build_dataloader` Ultralytics memakai `InfiniteDataLoader`, yang iteratornya persisten. Bila sebuah *loop* dihentikan dengan `break` (misalnya hanya 300 citra pertama untuk profil latensi) lalu *loader* yang sama dipakai lagi, iterasi berikutnya tidak mulai dari citra pertama. Akibatnya, hasil per citra tidak lagi sejajar dengan `dataset.labels`, dan GT yang dibangun dari label akan dipasangkan dengan deteksi citra lain. `rect=False` memastikan semua citra di-*letterbox* ke 640 × 640, sama dengan masukan statis *engine* TensorRT. Pemeriksaan

`expected_counts` menghentikan run bila dataset di Kaggle berbeda dari yang diharapkan (KITTI 1.297 val dan 1.000 uji; VisDrone 548 val dan 1.610 uji).

`image_id_of()` mengubah nama berkas menjadi ID COCO: `000014.png` menjadi 14 bila namanya angka, selain itu nama tanpa ekstensi. Aturan ini sama dengan `pred_to_json` Ultralytics.

4.2 Koordinat Asli dan Batas Objek Kecil

Metrik dihitung pada resolusi asli, bukan pada citra *letterbox*. Kategori luas COCO (kecil < 32^2 piksel) didefinisikan pada citra asli, dan citra KITTI (sekitar 1.242 × 375) diperkecil oleh *letterbox*. Mengukur luas pada citra *letterbox* akan menggeser banyak objek ke kategori kecil. Dua fungsi menangani konversi ini.

```
def small_area_of(ratio_pad):
    """Batas 'kecil' COCO (32² px asli) dalam piksel letterbox²."""
    (gh, gw), _ = ratio_pad
    return SMALL_PX ** 2 * float(gh) * float(gw)

def _scale_xyxy(b, ratio_pad, ori_shape):
    (gh, gw), (px, py) = ratio_pad
    b = b.clone()
    b[:, [0, 2]] = ((b[:, [0, 2]] - px) / gw).clamp(0, ori_shape[1])
    b[:, [1, 3]] = ((b[:, [1, 3]] - py) / gh).clamp(0, ori_shape[0])
    return b
```

`ratio_pad` dari `YOLODataset` berisi faktor skala per sumbu (gh, gw) dan *padding* (px, py). Kotak dikembalikan ke koordinat asli per sumbu, lalu dipotong ke batas citra, sama dengan `scale_boxes` Ultralytics. `small_area_of()` melakukan arah sebaliknya: batas 32^2 piksel asli diubah ke piksel *letterbox* kuadrat. Nilai ini dipakai oleh heuristik kepadatan (Subbab 7.4) dan latensi L2, yang bekerja pada kotak *letterbox*.

4.3 NMS dan NMS Cepat Bersertifikat

`nms_one()` menjalankan `non_max_suppression` stok dengan protokol validasi Ultralytics: conf 0,001, IoU 0,7, *multi-label*, sadar kelas, `max_det` 300 (VisDrone 500 di analisis), dan `max_nms` 30.000. Tanpa `fast_nms_k`, fungsinya persis NMS stok. VisDrone membutuhkan percepatan karena sebagian citranya menghasilkan hingga 30.000 kandidat pada conf 0,001. Bila `fast_nms_k = K` diberikan, NMS dijalankan dulu pada K kandidat berskor tertinggi, dan hasilnya hanya dipakai bila terbukti sama dengan NMS penuh.

```
k = int(fast_nms_k)
sc = y[0, 4:]                                   # (nc, N); kandidat multi-label = skor kelas > conf
cand = sc[sc > conf]
if cand.numel() <= k:                           # tidak ada yang dibuang -> sama dengan penuh
    NMS_STATS["penuh_langsung"] += 1
    return full()
kth = torch.topk(cand, k).values[-1]
x = non_max_suppression(y.clone(), conf, iou, nc=0, multi_label=True, agnostic=False, max_det=max_det,
                        max_nms=k)[0]
if len(x) == max_det and bool(x[max_det - 1, 4] > kth):
    NMS_STATS["cepat"] += 1
    return x
NMS_STATS["fallback"] += 1
return full()
```

Argumen kebenarannya sebagai berikut. NMS *greedy* memproses kandidat dari skor tertinggi, dan sebuah kotak hanya dapat ditekan oleh kotak lain yang berskor lebih tinggi. Karena itu, nasib kotak di antara K teratas tidak dipengaruhi kandidat di luar K teratas, sehingga kotak yang selamat di antara K teratas sama dengan versi penuh. Bila ada tepat `max_det` kotak selamat dan skor kotak ke-`max_det` lebih besar secara ketat daripada skor kandidat ke-K, semua kandidat yang dibuang berskor lebih rendah daripada seluruh keluaran. Kandidat itu tidak dapat menekan kotak keluaran dan tidak dapat

masuk `max_det` teratas, sehingga keluarannya identik. Bila syarat ini tidak terpenuhi, kode kembali ke NMS penuh. Satu-satunya celah adalah skor yang persis sama di antara kotak yang bertumpuk, karena urutan pemrosesannya dapat berbeda. `scripts/check_fast_nms.py` mengukur celah ini di val dalam FP32 dan FP16, dan notebook berhenti bila ada satu deteksi pun yang berbeda. Penghitung `NMS_STATS` dicetak di log; pada suite VisDrone tercatat 6.440 panggilan, dengan 6.346 penuh langsung, 94 cepat, dan 0 *fallback*.

`to_dets()` menjalankan `nms_one()`, mengembalikan kotak ke koordinat asli, lalu mengubahnya ke format COCO: array (n, 6) berisi x, y, w, h, skor, dan `category_id` = kelas + 1. `jdict()` mengubah daftar array itu menjadi daftar *dict* yang diterima `COCO.loadRes`.

Ada satu sifat NMS yang dipakai suite. NMS mengembalikan kotak terurut menurut skor, sehingga `d[:300]` dari keluaran ber-`max_det` 500 sama dengan keluaran ber-`max_det` 300. Uji `test_prefix_property_used_by_suite` memeriksa sifat ini, dan suite VisDrone memakainya untuk fit ulang gate dengan 300 deteksi teratas tanpa menjalankan NMS kedua kalinya.

4.4 Ground Truth COCO dan COCOeval

`build_coco_gt()` membangun GT COCO langsung dari `dataset.labels` pada resolusi asli. Label YOLO (cx, cy, w, h ternormalisasi) dikali lebar dan tinggi asli, `category_id` = kelas + 1, dan `area` = w × h dalam piksel asli, dengan `iscrowd` 0. Membangun GT dari label yang sama dengan yang dibaca *loader* menjamin GT dan deteksi berasal dari daftar citra yang sama.

`coco_eval()` menjalankan COCOeval lengkap dengan `imgIds` terurut dan `maxDets` dari konfigurasi, lalu mengembalikan dua hal: *dict* 12 metrik (`mAP50-95`, `mAP50`, `mAP75`, `mAP_small`, `mAP_medium`, `mAP_large`, `AR1`, `AR10`, `AR100`, `AR_small`, `AR_medium`, `AR_large`) dan objek COCOeval itu sendiri. Objek ini dibutuhkan karena `evalImgs`-nya dipakai untuk target gate dan semua metrik routing. Bila tidak ada deteksi sama sekali, fungsi membuat objek COCO kosong agar COCOeval tetap berjalan.

```
# pycocotools 2.0.x: stats[0] memakai maxDets=100 tertanam; hitung ulang dari maxDets[-1], area
'all'
prec = E.eval["precision"][:, :, :, 0, -1]
E.stats[0] = float(prec[prec > -1].mean()) if (prec > -1).any() else -1.0
```

Koreksi ini perlu untuk VisDrone. Di pycocotools, `stats[0]` (mAP50-95) dihitung dengan `maxDets=100` yang tertanam di kode, bukan dari parameter. Dengan `maxDets` [1, 10, 500], nilai 100 tidak ada di daftar, sehingga `stats[0]` menjadi −1. Kode menghitung ulang `stats[0]` dari kolom `maxDets` terakhir dan area *all*. Untuk KITTI dengan [1, 10, 100] hasilnya sama dengan bawaan. Nama `AR100` di *dict* tetap dipakai walaupun untuk VisDrone nilainya adalah AR pada 500 deteksi.

4.5 soft_small per Citra

`per_image_soft_small()` menghitung S_k pada Pers. (3) dari `evalImgs`: jumlah skor deteksi *true positive* pada IoU 0,5 untuk area *small*, yang tidak diabaikan, dalam batas `maxDet` terakhir, dijumlah atas kategori. Fungsi ini juga menghitung jumlah GT kecil yang tidak diabaikan per citra.

```
base = k * A * I + a_small * I
for i in range(I):
    e = E.evalImgs[base + i]
    if e is None:
        continue
    sc = np.asarray(e["dtScores"][:maxDet], float)
    tp = (np.asarray(e["dtMatches"][:, :maxDet]) > 0)[t50] \
         & ~np.asarray(e["dtIgnore"][:, :maxDet], bool)[t50]
    soft[i] += sc[tp].sum()
    n_gt[i] += int(np.count_nonzero(np.asarray(e["gtIgnore"]) == 0))
```

Rumus indeks `k * A * I + a_small * I + i` mengikuti tata letak `evalImgs` pycocotools: kategori di luar, lalu rentang area, lalu citra. `dtMatches[t50] > 0` berarti deteksi cocok dengan suatu GT pada

IoU 0,5, dan `dtIgnore` menandai deteksi yang diabaikan, yaitu deteksi yang cocok dengan GT yang diabaikan (GT di luar rentang area) atau deteksi tak cocok yang luasnya sendiri di luar rentang area. Keluaran diurutkan ulang dari urutan `imgIds` COCO ke urutan *loader* dengan permutasi `perm`. Δ soft_small adalah selisih fungsi ini untuk cabang A dan B; perhitungan yang sama ada di `Router.targets()` (`cocomix.py`).

5 Gate Post-hoc dan Kendali Anggaran: gate.py, budget.py

5.1 Cache Fitur dan Deteksi

Gate dilatih setelah detektor dibekukan dan tidak menyentuh bobot detektor. Langkah pertama adalah `cache_split()`, yang menjalankan satu *forward* per citra dan menyimpan semua yang dibutuhkan tahap berikutnya.

```
@torch.no_grad()
def cache_split(model, loader, device, nms, dets=True, log_every=1000, log=print):
    a, b_ = BranchA(model).eval(), BranchB(model).eval()
    out = {k: [] for k in ("file", "image_id", "feat", "detA", "detB")}
    for i, b in enumerate(loader):
        assert b["img"].shape[0] == 1, "cache_split butuh batch 1"
        im = b["img"].to(device).float() / 255.0
        p3, p2, p4, p5 = model.trunk(im)
        out["feat"].append(pool4(p3, p2, p4, p5)[0].cpu().numpy())
        if dets:
            rp, osh = b["ratio_pad"][0], b["ori_shape"][0]
            out["detA"].append(to_dets(a(p3, p2, p4, p5), rp, osh, **nms))
            out["detB"].append(to_dets(b_(p3, p4, p5), rp, osh, **nms))
        # ... file, image_id
```

Trunk dijalankan sekali, lalu keluarannya dipakai untuk fitur dan untuk kedua cabang. Ini sama dengan yang terjadi di *engine*, dan menjamin bahwa fitur gate dan deteksi kedua cabang berasal dari *forward* yang sama. Untuk val, `dets=False` dipakai karena hanya skor yang dibutuhkan untuk ambang awal, sehingga NMS dan COCOeval dilewati. Suite memakai versi yang lebih luas dengan nama sama di `suite.py`; versi itu juga menyimpan skor heuristik, fitur *trunk* Vanilla, serta deteksi Vanilla dan P2-Static.

5.2 Target dan Fit Gate

`soft_small_delta()` membangun GT, menjalankan COCOeval untuk deteksi A dan B, lalu menghitung Δ soft_small = S_A − S_B per citra dengan `per_image_soft_small()`. Fungsi ini juga memeriksa bahwa jumlah GT kecil per citra sama untuk kedua cabang, sebagai pemeriksaan bahwa kedua evaluasi memakai GT dan urutan yang sama. `fit_gate()` lalu melatih gate.

```
def fit_gate(feat, delta, C=0.001, max_iter=5000):
    X = np.asarray(feat, np.float64)
    y = (np.asarray(delta) > 0).astype(int)
    assert 0 < y.sum() < len(y), "target gate hanya satu kelas"
    pipe = make_pipeline(StandardScaler(), LogisticRegression(C=C, max_iter=max_iter)).fit(X, y)
    sc, lr = pipe[0], pipe[1]
    scale = np.where(sc.scale_ == 0, 1.0, sc.scale_)
    w = lr.coef_[0] / scale
    b = float(lr.intercept_[0] - np.sum(lr.coef_[0] * sc.mean_ / scale))
    ref = pipe.decision_function(X[:256])
    assert np.allclose(X[:256] @ w + b, ref, atol=1e-6 * max(1.0, np.abs(ref).max())), \
        "lipatan scaler + logreg salah"
```

Labelnya adalah y = 1[Δ > 0] [Pers. (3)]. Citra dengan Δ = 0 (misalnya citra tanpa objek kecil, atau objek kecil yang dideteksi sama baiknya) diberi label 0, artinya rute B sudah cukup. *Pipeline* scikit-learn adalah `StandardScaler` diikuti `LogisticRegression` dengan regularisasi L2, C = 0,001, *solver* lbfgs bawaan, dan `max_iter` 5.000. Fit dilakukan dalam float64. Pemeriksaan `assert 0 < y.sum() < len(y)` mencegah fit dengan satu kelas, yang dapat terjadi pada subset kecil di ablasi biaya label.

5.3 Melipat Scaler dan Regresi Logistik

Standardisasi dan regresi logistik keduanya afin, sehingga komposisinya juga afin [Pers. (4)]. Dengan μ dan σ statistik `StandardScaler` serta (w, b) koefisien regresi logistik, skornya adalah Σ_j w_j (f_j − μ_j)/σ_j + b, yang sama dengan Σ_j (w_j/σ_j) f_j + (b − Σ_j w_j μ_j/σ_j). Itulah `w` dan `b` di kode. Kolom dengan σ = 0 (fitur konstan) diberi σ = 1, sama dengan perilaku `StandardScaler`. Lipatan ini diperiksa langsung: skor 256 citra pertama dari (w, b) yang dilipat harus sama dengan `decision_function` *pipeline* scikit-learn dalam toleransi relatif 10^{-6}. Hasilnya, gate di *engine* hanya berupa *pooling* dan satu perkalian vektor 960 dimensi.

`save_gate()` menyimpan (w, b) dan informasi fit ke `gate.json`. Bobot ditulis sebagai float64 dengan representasi penuh, sehingga dimuat ulang bit-identik. `fit_gate.py` menambahkan `thr0`, ρ, sha256 bobot detektor, target, split, serta statistik COCO train cabang A dan B. `score_np()` menghitung skor di numpy (fitur FP32 dikali bobot float64), dan inilah skor acuan PyTorch FP32 di evaluasi. `GateHead` adalah versi `nn.Module`-nya untuk *engine*: bobot disimpan sebagai *buffer* FP32 dan fitur dipaksa FP32.

```
class GateHead(nn.Module):
    def __init__(self, w, b):
        super().__init__()
        self.register_buffer("w", torch.as_tensor(np.asarray(w), dtype=torch.float32))
        self.register_buffer("b", torch.tensor([float(b)], dtype=torch.float32))

    def forward(self, p3, p2, p4, p5):
        return pool4(*(t.float() for t in (p3, p2, p4, p5))) @ self.w + self.b
```

Skor ini berbentuk *logit*, bukan probabilitas. Keputusan tidak memakai ambang 0 (setara probabilitas 0,5), melainkan ambang dari kendali anggaran. Langkah terakhir `fit_gate.py` menghitung skor val dan `thr0 = thr0_from_val(s_va, ρ)`, yaitu kuantil ke-(1 − ρ) skor val, yang menjadi ambang awal B2.

5.4 Kendali Anggaran B2

`budget.py` mengimplementasikan Pers. (5) dalam dua bentuk yang hasilnya harus sama: versi *batch* untuk analisis luring dan versi *streaming* untuk *runtime*.

```
def b2_thresholds(scores, rho, window, min_hist, thr0):
    s = np.asarray(scores, np.float64)
    out = np.empty(len(s), np.float32)
    for t in range(len(s)):
        hist = s[max(0, t - window):t]
        out[t] = np.float32(np.quantile(hist, 1 - rho)) if len(hist) >= min_hist else
np.float32(thr0)
    return out

def b2_decide(scores, rho, window, min_hist, thr0):
    thr = b2_thresholds(scores, rho, window, min_hist, thr0)
    return np.asarray(scores, np.float32) > thr, thr
```

Riwayat untuk citra ke-t hanya memuat skor citra t − W sampai t − 1. Skor citra ke-t sendiri tidak ikut, sehingga keputusan tidak memakai informasi masa depan atau dirinya sendiri. Selama riwayat kurang dari 20 skor, ambang awal `thr0` dipakai. Perbandingan dilakukan dalam float32 (`np.float32(score) > np.float32(thr)`), karena di *engine* operator `Greater` membandingkan dua tensor FP32. Dengan begitu keputusan analisis dan *engine* sama persis untuk skor dan ambang yang sama.

`B2Controller` adalah versi *runtime*: `threshold()` dipanggil sebelum *engine* dijalankan, lalu `push(score)` setelah skor keluar dari *engine*. Metode `decide()` menggabungkan keduanya untuk evaluasi luring per citra. Uji `test_b2_matches_reference_and_streaming` memastikan versi *streaming* identik dengan salinan apa adanya dari implementasi Tahap 4.

5.5 Kuantil Jendela Terurut

Implementasi naif menghitung `np.quantile` atas 200 skor untuk setiap citra, termasuk membuat array baru dari `deque`. Di CPU Kaggle, biaya ini terukur 0,099–0,104 ms per citra, sedangkan versi efisien hanya 0,0012–0,0013 ms. Pada skala latensi sekitar 2,4 ms, selisih ini cukup besar untuk menggeser fraksi campuran *f* di Pers. (7). `SortedWindowQuantile` menyimpan jendela dalam dua struktur: `deque` untuk urutan masuk dan daftar terurut untuk kuantil. Skor baru disisipkan dengan `bisect.insort` dan skor tertua dibuang dengan pencarian biner.

```
def quantile(self, q):
    n = len(self.s); pos = q * (n - 1); lo = int(pos); fr = pos - lo
    a, b = self.s[lo], self.s[min(lo + 1, n - 1)]
    # rumus lerp numpy (np.quantile 'linear'): a + d·t bila t < 0,5,
    # selain itu b − d·(1 − t) -> bit-identik
    return a + (b - a) * fr if fr < 0.5 else b - (b - a) * (1 - fr)
```

Rumus interpolasi di baris terakhir sengaja meniru rumus *lerp* internal numpy, yang memakai dua bentuk tergantung pecahan posisi. Secara aljabar keduanya sama, tetapi dalam aritmetika *floating point* hasilnya dapat berbeda pada bit terakhir. Bila rumusnya berbeda dari numpy, ambang efisien dapat berbeda satu ULP dari `np.quantile`, dan untuk skor yang tepat berada di ambang keputusannya dapat berbalik. Uji `test_b2_efficient_bit_identical_to_np_quantile` membandingkan kedua implementasi pada 5.000 skor dengan empat nilai ρ, termasuk skor kembar yang disisipkan sengaja, dan menuntut ambang serta keputusan identik. Di kode dan berkas hasil, versi naif disebut “B2 terkunci” (implementasi Tahap 4 yang dikunci) dan versi efisien disebut “B2 efisien”. Di tesis keduanya disebut kendali naif dan kendali efisien.

`b2_host_overhead()` mengukur biaya host kedua implementasi pada 4.000 skor acak dengan jendela yang sama, setelah memastikan nilai kuantilnya sama dalam 10^{-12}. Hasilnya dipakai sebagai T_τ di Pers. (6) untuk latensi komposit.

6 Deployment A3: export.py, trt.py, evaluate.py

6.1 Antarmuka Engine

Engine A3 adalah satu graf ONNX yang dibangun menjadi satu *engine* TensorRT. Antarmukanya ditulis di kepala `export.py`, dan semua kode lain (evaluasi, suite, profil, L2, A12) bergantung pada antarmuka ini.

```
input : image (1, 3, 640, 640) FP32 [0, 1], mode (1,) FP32, thr (1,) FP32
cond  : mode < 0 ? (score > thr) : (mode > 0.5)          mode −1 = ikut gate, 1 = paksa A, 0 =
paksa B
output: y (1, 4 + nc, 34000) — rute B di-pad nol dari 8400 (skor kelas 0 -> dibuang ambang conf
NMS),
        use_a (1,), score (1,)
```

Ambang adalah **masukan** *engine* dan skor adalah **keluaran** *engine*. Dengan susunan ini, B2 berjalan sepenuhnya di host tanpa *engine* kedua: host menghitung ambang dari riwayat, menjalankan *engine*, lalu memasukkan skor yang keluar ke riwayat. Masukan `mode` memungkinkan rute dipaksa dengan *engine* yang sama, sehingga latensi rute A dan rute B diukur pada *engine* yang persis sama dengan router.

6.2 Stage 1 dan Presisi

Ekspor dimulai dengan memotong model menjadi tiga modul: `Stage1` (citra → skor dan empat fitur), `BranchA`, dan `BranchB`.

```
class Stage1(nn.Module):
    def __init__(self, model, gate):
        super().__init__()
        self.m = model
        self.gate = GateHead(gate["w"], gate["b"])
        self._dt = next(model.parameters()).dtype

    def forward(self, x):
        p = [t.float() for t in self.m.trunk(x.to(self._dt))]
        return (self.gate(*p), *p)

class IOCast(nn.Module):
    def forward(self, *xs):
        out = self.m(*[x.half() for x in xs])
        return tuple(o.float() for o in out) if isinstance(out, (tuple, list)) else out.float()
```

Untuk ekspor FP16, salinan model diubah ke FP16. `Stage1` menjalankan *trunk* dalam tipe model, lalu mengubah keempat fitur ke FP32 sebelum *pooling* dan Gemm gate. Bila *pooling* dan perkalian gate dijalankan dalam FP16, pembulatan bobot gate dan akumulasi 960 suku dapat menggeser skor cukup jauh untuk membalik keputusan citra yang skornya dekat ambang. Stage 2 dibungkus `IOCast`, yang mengubah masukan FP32 menjadi FP16 dan keluarannya kembali FP32. Jadi semua tensor antar-tahap dan semua masukan/keluaran *engine* adalah FP32, sedangkan komputasi berat berjalan FP16. TensorRT dibangun dengan jaringan *strongly typed*, sehingga presisi setiap operasi mengikuti tipe di ONNX dan tidak dipilih ulang oleh *builder*.

Sebelum *tracing* FP16, `export_a3()` menjalankan cabang FP16 sekali di luar *tracing*. Kepala `Detect` menghitung *anchor* hanya ketika bentuk masukan berubah, sehingga panggilan ini membuat *anchor* FP16 sudah tersedia dan ikut ter-*trace* sebagai konstanta. ONNX FP32 selalu diekspor di CPU untuk verifikasi, sedangkan FP16 membutuhkan GPU karena operator *half* di CPU tidak lengkap.

6.3 Membangun Operator If

`build_if()` menggabungkan tiga berkas ONNX terpisah (`s1`, `s2a`, `s2b`) menjadi satu graf. Langkah pertamanya, `_prefixed_graph()`, menyalin graf Stage 2A dan 2B dengan awalan `A__` dan `B__` pada semua nama *node*, tensor, dan *initializer*, kecuali empat nama fitur dari Stage 1. Awalan ini diperlukan karena dua graf yang diekspor terpisah memakai nama yang sama (misalnya nama *node* konvolusi layer 19), sedangkan ONNX menuntut setiap nama tensor unik di seluruh model, termasuk di dalam subgraf. Empat nama fitur tidak diberi awalan karena subgraf membacanya dari *scope* luar.

```
pad = numpy_helper.from_array(np.zeros((1, no, n_a - n_b), np.float32), "B__pad_zeros")
nb.append(helper.make_node("Concat", [yb, "B__pad_zeros"], ["B__y_pad"], axis=2,
name="B__pad_concat"))
then_g = helper.make_graph(na, "rute_A", [], [vi(ya)], initializer=ia)
else_g = helper.make_graph(nb, "rute_B", [], [vi("B__y_pad")], initializer=ib + [pad])
nodes = list(g1.node) + [
    # ... konstanta c_zero = 0, c_half = 0,5, c_ax0 = [0]
    helper.make_node("Less", ["mode", "c_zero"], ["is_auto"]),
    helper.make_node("Greater", ["score", "thr"], ["gate_on"]),
    helper.make_node("Cast", ["gate_on"], ["gate_f"], to=TensorProto.FLOAT),
    helper.make_node("Where", ["is_auto", "gate_f", "mode"], ["sel_f"]),
    helper.make_node("Greater", ["sel_f", "c_half"], ["sel"]),
    helper.make_node("Squeeze", ["sel", "c_ax0"], ["cond"]),
    helper.make_node("If", ["cond"], ["y"], then_branch=then_g, else_branch=else_g,
name="rute_if"),
    helper.make_node("Cast", ["sel"], ["use_a"], to=TensorProto.FLOAT),
]
```

Urutan *node* keputusan dapat dibaca seperti ini. `is_auto` bernilai benar bila mode negatif (−1). `gate_on` adalah hasil perbandingan skor dengan ambang, diubah ke 0 atau 1. `Where` memilih nilai itu bila mode otomatis, atau nilai mode itu sendiri (0 atau 1) bila dipaksa. Hasilnya dibandingkan dengan

0,5 sehingga menjadi *boolean*, lalu `Squeeze` mengubahnya menjadi skalar karena operator `If` membutuhkan kondisi skalar. `use_a` adalah kondisi yang sama dalam bentuk FP32, dikeluarkan supaya host tahu rute yang diambil.

Kedua subgraf `If` wajib punya bentuk keluaran yang sama. Karena itu keluaran rute B (1, 6, 8.400) disambung dengan tensor nol (1, 6, 25.600) menjadi (1, 6, 34.000). *Anchor padding* tidak menghasilkan deteksi: NMS hanya memakai kandidat dengan skor kelas maksimum di atas conf (0,001), sedangkan skor *padding* adalah 0. Uji `test_nms_route_b_cut` memeriksa bahwa NMS atas 34.000 *anchor* rute B memberi kotak yang sama dengan NMS atas 8.400 *anchor* aslinya. Biaya *padding* ada di sisi lain: rute B menyalin 816 KB keluaran ke host, bukan 202 KB seperti Vanilla. Inilah yang diselidiki profil per segmen dan A12 (Subbab 8.3 dan 8.5).

Graf akhir diperiksa dengan `onnx.checker.check_model` sebelum disimpan. *Opset* dan versi IR disalin dari graf Stage 1.

6.4 Verifikasi ONNX

`verify_a3()` membandingkan graf FP32 di ONNX Runtime dengan modul PyTorch FP32 pada beberapa citra val. Untuk setiap citra, empat kombinasi diuji: paksa A dengan ambang 10^9, paksa B dengan ambang -10^9, otomatis dengan ambang skor − 0,5 (harus A), dan otomatis dengan ambang skor + 0,5 (harus B). Dua kombinasi pertama memeriksa bahwa mode paksa mengalahkan gate. Dua yang terakhir memeriksa kedua arah perbandingan dengan ambang. Rute harus benar di semua kombinasi, dan selisih relatif y serta skor harus di bawah 10^{-4}. Pada hasil final, selisih y paling besar 3×10^{-6} dan selisih skor gate paling besar $4{,}8 \times 10^{-5}$.

Verifikasi ini hanya mencakup graf FP32. Kesetaraan *engine* FP16 diperiksa di tempat lain: suite membandingkan keputusan FP16 dengan keputusan PyTorch FP32 pada seluruh data uji, dan hasilnya 99,8–99,9% sama.

6.5 Engine TensorRT

`build_engine()` mem-*parse* ONNX, memasang batas *workspace* 1 GB, lalu membangun *engine* dengan bendera `STRONGLY_TYPED` bila versi TensorRT menyediakannya. Bendera `FP16` hanya dipasang untuk TensorRT lama yang tidak mengenal *strongly typed*. `A3Engine` membungkus *engine* untuk evaluasi.

```python
@torch.no_grad()
def infer(self, img, thr, mode=-1.0):
    self.stream.wait_stream(torch.cuda.current_stream(self.dev))
    with torch.cuda.stream(self.stream):
        self.buf["image"].copy_(img, non_blocking=True)
        self.buf["thr"].fill_(float(thr)); self.buf["mode"].fill_(float(mode))
        assert self.ctx.execute_async_v3(self.stream.cuda_stream), "execute_async_v3 gagal"
        self.h_use.copy_(self.buf["use_a"], non_blocking=True)
        self.h_score.copy_(self.buf["score"], non_blocking=True)
    self.stream.synchronize()
    return self.buf["y"], bool(float(self.h_use[0]) > 0.5), float(self.h_score[0])
```

Semua *buffer* adalah tensor CUDA PyTorch yang dialokasikan sekali, dan alamatnya didaftarkan ke konteks TensorRT sekali, sehingga tidak ada alokasi per panggilan dan tidak perlu `pycuda`. *Engine* berjalan di *stream* terpisah. `wait_stream` memastikan *stream engine* menunggu operasi di *stream* PyTorch, misalnya citra yang baru dikonversi atau `clone()` keluaran panggilan sebelumnya. Penanda rute dan skor disalin ke memori *pinned* di host. Perlu diingat bahwa `y` yang dikembalikan adalah *buffer* yang sama untuk setiap panggilan. Pemanggil yang memanggil *engine* lebih dari sekali per citra harus menyalinnya dulu, dan itulah sebabnya `run_split()` memanggil `yA.clone()`.

6.6 Evaluasi Satu Split

`evaluate.py` adalah evaluasi cepat sistem final pada satu split, dengan tiga *backend*. *Backend* `pytorch` memakai detektor FP32 dan skor gate float64 numpy; ini acuan yang sama dengan pemeriksaan silang FP32 di suite. *Backend* `ort` memakai graf A3 FP32 di ONNX Runtime CPU untuk uji lokal. *Backend* `trt` memakai *engine* A3 TensorRT, yaitu sistem yang di-*deploy*.

```
thr = ctl.threshold()
if backend == "pytorch":
    p3, p2, p4, p5 = model.trunk(im)
    s = float(score_np(pool4(p3, p2, p4, p5).cpu().numpy(), gate)[0])
    yA, yB = a(p3, p2, p4, p5), b_(p3, p4, p5)
    use = bool(np.float32(s) > thr)
    yR = yA if use else yB
else:
    yA, _, sA = eng.infer(im, thr, 1.0); yA = yA.clone()
    yB, _, _ = eng.infer(im, thr, 0.0); yB = yB.clone()
    yR, use, s = eng.infer(im, thr, -1.0)
    assert use == bool(np.float32(s) > thr), "keputusan engine != perbandingan host"
    assert abs(s - sA) <= 1e-6 * (1 + abs(s)), "skor gate berbeda antar-mode"
ctl.push(s)
```

Untuk *backend* `ort` dan `trt`, setiap citra dijalankan tiga kali: paksa A, paksa B, dan otomatis. Dua `assert` memeriksa bahwa keputusan *engine* sama dengan perbandingan float32 di host, dan bahwa skor gate tidak bergantung pada mode (Stage 1 sama untuk ketiga mode). `metrics()` lalu menghitung metrik COCO untuk A paksa, B paksa, router, dan rata-rata 20 routing acak pada aktivasi yang sama. Ada satu `assert` lagi: metrik router harus sama persis dengan metrik campuran deteksi A/B menurut keputusan router. Dengan begitu, keluaran `If` terbukti sama dengan rute yang dipaksa.

Routing acak di `evaluate.py` memakai *seed* 0 dan 20 undian, berbeda dari suite. Skrip ini adalah pemeriksaan cepat. Angka tesis berasal dari suite, dan selisih keduanya pada hasil final adalah paling besar 0,0008 untuk mAP_small dan 0,002 untuk aktivasi.

7 Statistik dan Evaluasi Final: cocomix.py, baselines.py, suite.py

Bab ini menjelaskan bagian kode yang menghasilkan angka utama tesis. `cocomix.py` murni numpy di atas `evalImgs` pycocotools dan tidak bergantung pada model. `baselines.py` menyediakan pembanding. `suite.py` mengatur semuanya menjadi protokol evaluasi final yang dikunci.

7.1 Metrik Routing dari evalImgs: CocoBootstrap, CocoMix, Router

Kunci seluruh analisis routing adalah satu sifat COCOeval. Tahap `evaluate()` mencocokkan deteksi dengan GT **per citra dan per kategori**, sedangkan hanya tahap `accumulate()` yang bersifat global. Akibatnya, untuk routing apa pun, hasil `evaluate()` atas deteksi ter-*route* sama dengan memilih, per citra, hasil `evaluate()` cabang A atau cabang B. Yang perlu dihitung ulang hanya `accumulate()`. Dengan sifat ini, metrik ribuan routing (router, acak, campuran, ablasi, *resample bootstrap*) dapat dihitung dari dua kali COCOeval saja.

`CocoBootstrap` mengambil dari setiap sel `evalImgs` (kategori × area × citra) empat hal: skor deteksi sampai `maxDet` terakhir, matriks kecocokan (T ambang IoU × D deteksi), matriks *ignore*, dan jumlah GT yang tidak diabaikan. Metode `_precision(idx, area)` lalu mereplikasi `accumulate()` untuk subset citra `idx`, yang boleh berisi citra berulang.

```
scores = np.concatenate([e[0] for e in E])
order = np.argsort(-scores, kind="mergesort")
dtm = np.concatenate([e[1] for e in E], axis=1)[:, order]
dtig = np.concatenate([e[2] for e in E], axis=1)[:, order]
tp = np.cumsum(dtm & ~dtig, axis=1, dtype=float)
fp = np.cumsum(~dtm & ~dtig, axis=1, dtype=float)
# ...
rc = tp / npig
pr = tp / (fp + tp + np.spacing(1))
pr = np.maximum.accumulate(pr[:, ::-1], axis=1)[:, ::-1]
for t in range(self.T):
    ii = np.searchsorted(rc[t], self.R, side="left")
    ok = ii < nd
    q[t, ok] = pr[t, ii[ok]]
```

Setiap baris di atas sama dengan langkah di pycocotools: pengurutan stabil (`mergesort`) menurut skor menurun, jumlah kumulatif TP dan FP yang tidak diabaikan, *recall* dan presisi, interpolasi presisi (maksimum dari kanan), lalu pengambilan presisi pada 101 ambang *recall*. Kategori tanpa GT dilewati, sama dengan nilai −1 yang dikecualikan COCOeval. `metrics()` mengembalikan tiga metrik yang diuji: mAP50-95 (rata-rata atas kategori, 10 ambang IoU, dan 101 ambang *recall*), mAP50, dan mAP_small.

`CocoMix` adalah `CocoBootstrap` yang selnya dipilih per citra dari A atau B menurut *mask*. `Router` menyimpan dua `CocoBootstrap`, permutasi dari urutan *loader* ke urutan `imgIds`, dan metode `targets()`. Metode ini menghitung tiga target per citra untuk selisih A − B: `soft_small` (target final), `soft_all` (semua ukuran), dan `net25_all` (TP − FP pada skor ≥ 0,25), ditambah jumlah GT kecil dan GT total. Dua target terakhir dipakai ablasi target gate.

Sifat “komposisi per citra sama dengan COCOeval langsung” tidak hanya diklaim. Di `analyze()`, metrik CocoMix untuk router, heuristik, A, Vanilla, dan P2-Static dibandingkan dengan COCOeval yang dijalankan langsung atas deteksi ter-*route*, dan selisihnya harus di bawah 10^{-9}. `test_router_union_fast_equal_cocoeval` memeriksa hal yang sama pada data sintetis.

7.2 Bootstrap Berpasangan: UnionBoot, FastUnionBoot, boot_engine

Dengan B = 1.000 *resample* dan sekitar 30 routing per model, `CocoBootstrap` terlalu lambat karena mengurutkan ulang deteksi untuk setiap *resample*. `UnionBoot` mengurutkan gabungan deteksi kedua sumber **sekali** per (kategori, area), dengan kunci (−skor, citra, sumber, posisi). Kunci ini sama dengan urutan `accumulate()` untuk routing apa pun pada himpunan penuh, karena untuk skor yang sama COCOeval mengurutkan menurut urutan citra lalu posisi deteksi, dan pada satu citra hanya satu sumber yang aktif. Deteksi yang diabaikan di semua ambang IoU dibuang sejak awal.

```
def metrics(self, use_a_sorted, idx):
    c = np.bincount(idx, minlength=self.I)
    for (k, area), (tp, fp, img, src, npig) in self.cells.items():
        npg = int((c * npig).sum())
        # ...
        w = c[img] * ((src == 0) == use_a_sorted[img])
        sel = np.flatnonzero(w)
        ww = w[sel].astype(np.int32)
        tpc = np.cumsum(tp[:, sel] * ww, axis=1, dtype=np.int32).astype(float)
        fpc = np.cumsum(fp[:, sel] * ww, axis=1, dtype=np.int32).astype(float)
```

Sebuah *resample* diwakili oleh multiplisitas c_i, yaitu berapa kali citra i terambil. Setiap deteksi diberi bobot w = c_i bila sumbernya aktif untuk citra itu, dan 0 bila tidak. Jumlah GT juga dikali c_i. Bobot c_i setara dengan c_i salinan citra yang sama, karena salinan suatu deteksi punya skor dan status

TP/FP yang sama. Bila salinan dimasukkan satu per satu, titik-titik di tengah deretan salinan itu tidak pernah menentukan presisi terinterpolasi: dalam deretan TP presisi naik sehingga maksimum dari kanan sudah dicapai di ujung deretan, dan deretan FP tidak menambah *recall* sehingga tidak pernah menjadi titik pertama yang mencapai suatu ambang *recall*. `FastUnionBoot` memakai sifat yang sama lebih jauh. Presisi terinterpolasi hanya perlu dievaluasi di posisi TP, dan penyebutnya dihitung dari jumlah kumulatif bobot deteksi yang tidak diabaikan di semua ambang IoU, ditambah koreksi jarang untuk deteksi yang diabaikan hanya di sebagian ambang. Hasilnya sekitar tujuh kali lebih cepat di VisDrone, dan uji `test_cocomix.py` menuntut hasilnya sama dengan `UnionBoot` dan COCOeval.

`boot_engine()` membuat indeks *resample* `rng(seed).integers(0, I, (n_boot, I))` sekali, lalu memakai indeks yang sama untuk **semua** routing. Karena itu setiap uji berpasangan: selisih dua routing dihitung pada *resample* yang sama, sehingga variasi komposisi data uji ikut terkurangi. Karena *seed* (`boot_seed` 20261016) dan jumlah citra sama, indeks *resample* juga sama untuk ketiga *seed* KITTI, dan ini yang membuat *pooling* sah. Estimasi titik dihitung pada himpunan penuh. *Resample* dibagi ke beberapa proses `fork`; bila melewati batas waktu, perhitungan diulang secara serial.

7.3 Uji, Nilai p, Holm, Korelasi Parsial, Pooling, dan Klaster

`boot_tests()` menerima daftar uji berbentuk (label, a, [b_1, …, b_m]). Untuk setiap metrik, statistiknya adalah metrik a dikurangi rata-rata metrik b_1…b_m. Untuk pembanding acak, b adalah lima undian acak, sehingga yang dibandingkan adalah router terhadap rata-rata lima routing acak, pada setiap *resample*. Selang kepercayaan 95% adalah persentil 2,5 dan 97,5 dari sebaran *resample*.

```
def _pval(s):
    return float(min(1.0, max(2 * min((s <= 0).mean(), (s >= 0).mean()), 1.0 / len(s))))
```

Nilai p dua sisi adalah dua kali proporsi *resample* di sisi nol yang lebih kecil, dengan batas bawah 1/B (0,001 untuk B = 1.000) dan batas atas 1. Batas bawah ini mencegah p = 0, yang tidak dapat diklaim dengan 1.000 *resample*. `holm()` adalah koreksi Holm *step-down*: nilai p diurutkan naik, yang ke-r dikali (m − r), lalu diambil maksimum kumulatifnya agar urutannya monoton.

`partial_spearman_p()` menghitung korelasi Spearman parsial yang dipakai kriteria H3. Semua variabel diubah ke peringkat, peringkat x dan y masing-masing diregresikan terhadap peringkat variabel kontrol (dengan *intercept*), lalu korelasi Pearson kedua residu diambil. Nilai p memakai statistik t dengan derajat bebas n − 2 − k, dengan k jumlah variabel kontrol.

`pooled_tests()` merata-ratakan selisih antar-model **per *resample***, lalu mengambil persentil dari rata-rata itu. Cara ini sah hanya bila model berbagi data uji dan indeks *resample*, seperti tiga *seed* KITTI. `cluster_index()` membuat indeks *bootstrap* klaster: klaster (video VisDrone atau rekaman KITTI) diambil dengan pengembalian, lalu posisi citra dalam klaster terpilih digabung. Indeks klaster ini dimasukkan ke `boot_engine()` lewat argumen `idx`, sehingga semua kode lain tidak berubah.

7.4 Pembanding: Vanilla, P2-Static, Heuristik, Kaskade

`load_static()` memuat Vanilla dan P2-Static stok dari `last.pt` lewat `YOLO(...)` dalam FP32, dan menghentikan run bila jumlah kelasnya berbeda dari dataset (misalnya bila *checkpoint* pembanding berasal dari dataset lain). `StaticOut` mengambil tensor `y` dari keluaran model. `static_with_feats()` menjalankan Vanilla sambil menyimpan keluaran layer 15, 2, 12, dan 9. Indeks ini sama dengan *trunk* dua cabang, karena layer 0–15 Vanilla punya struktur yang sama. Fitur ini dipakai gate kaskade.

Heuristik kepadatan objek kecil adalah pembanding tanpa label dan tanpa fit. `P3Density` memakai kepala P3 `Detect_B` (`cv2[0]`, `cv3[0]`, `dfl`) pada `p3_neck` di Stage 1, menghitung *anchor* dengan skor kelas maksimum di atas 0,25 (dibandingkan dalam *logit*), dan menghitung yang kotak hasil *decode*-nya lebih kecil dari batas objek kecil *letterbox*.

```
def counts(self, p3, small_area):
    box, cls = self.cv2(p3), self.cv3(p3)
    b, _, h, w = cls.shape
    hit = cls.amax(1).reshape(b, h * w) > self.logit_thr
    d = self.dfl(box.reshape(b, 4 * self.reg_max, h * w)).float() * self.stride
    area = (d[:, 0] + d[:, 2]) * (d[:, 1] + d[:, 3])
    small = area < small_area.float().reshape(-1, 1)
    return hit.float().sum(1), (hit & small).float().sum(1)

def combine(self, n_all, n_small):
    return torch.log(1.0 + n_small) + self.eps * torch.log(1.0 + n_all)
```

Skornya log(1 + n_kecil) + 0,01 · log(1 + n_semua). Suku kedua hanya pemecah seri, karena skor yang murni cacahan punya banyak nilai kembar. `S1Heur` adalah Stage 1 versi heuristik dengan masukan tambahan `small_area`, dan suite mengekspornya sebagai *engine* A3 kedua (`a3h`) supaya latensi heuristik diukur pada TensorRT juga. Bila *engine* itu gagal dibangun, job tidak dihentikan karena heuristik hanya pembanding, dan skor heuristik diambil dari PyTorch FP32.

Pembanding kaskade bergate Vanilla → P2-Static memakai gate yang dilatih dengan cara sama, tetapi dengan fitur *trunk* Vanilla dan target Δ soft_small P2-Static − Vanilla. Latensinya tidak diukur sebagai sistem nyata, melainkan dimodelkan: rute A = Stage 1 dengan gate + P2-Static utuh − salin citra (citra sudah di GPU), dan rute B = Vanilla. Aktivasi kaskade dipilih sehingga latensi modelnya sama dengan router, lalu citra dengan skor kaskade tertinggi dirutekan ke P2-Static. Karena pemilihan top-ρ memakai sebaran skor uji, pembanding ini bersifat transduktif dan latensinya optimistis.

7.5 Suite Tahap A: Cache, Cek Gate, Ekspor

`stage_a()` berjalan di PyTorch dan menyiapkan semua yang dibutuhkan job TensorRT. Urutannya adalah sebagai berikut.

Pertama, *loader* train, val, dan uji dibuat dengan *batch* 1. Detektor dimuat, *seed* di *checkpoint* dicocokkan dengan *seed* run, dan `check_stage_equivalence()` memastikan *trunk* + `BranchA`/`BranchB` sama dengan *forward* penuh dalam toleransi 10^{-5}. Kedua, *cache* train dijalankan untuk detektor dua cabang (dan untuk Vanilla serta P2-Static bila kaskade aktif). Target gate dihitung dari 300 deteksi teratas (`gate_max_det`) walaupun analisis VisDrone memakai 500, karena gate di-fit dengan NMS ber-`max_det` 300.

Ketiga, `gate.json` dimuat, lalu gate di-fit ulang dari *cache* train sebagai pemeriksaan silang.

```
gj = load_gate(spec["gate"])
refit = fit_gate(Ctr["feat"], TG_tr["soft_small"], cfg["gate_C"])
w_rel = float(np.abs(refit["w"] - gj["w"]).max() / (np.abs(gj["w"]).max() + 1e-12))
# ...
assert w_rel < 1e-4, f"gate.json berbeda dari fit ulang (w rel {w_rel:.2e}): checkpoint/data/C
berbeda?"
```

Yang dipakai selanjutnya tetap bobot dari `gate.json`; fit ulang hanya membuktikan bahwa `gate.json` berasal dari *checkpoint*, data, dan C yang sama. Pada hasil final, selisihnya 0 di keempat model. Keempat, *cache* val (tanpa NMS) menghasilkan skor gate dan heuristik, dan `thr0` dihitung ulang lalu dibandingkan dengan `gate.json`. Kelima, *cache* uji menyimpan deteksi A, B, Vanilla, dan P2-Static, skor gate FP32, dan skor heuristik. Keenam, GFLOPs setiap komponen dihitung, lalu `export_suite()` mengekspor delapan graf (Stage 1 gate dan heuristik, Stage 2A dan 2B, Vanilla, P2-Static, A3 gate, A3 heuristik) dalam FP32 dan FP16, dan `verify_suite()` memeriksa semuanya di ONNX Runtime. Semua hasil disimpan di `stageA.pkl`.

7.6 Job Inferensi TensorRT

Bagian TensorRT dijalankan oleh `job_main()` di proses Python terpisah, dipanggil `run_subprocess()` dengan batas waktu (`trt_timeout_s` 10.800 detik). Dari kodenya, pemisahan ini memberi tiga hal: proses dapat dihentikan paksa bila macet, 80 baris terakhir lognya disimpan ke `job_log_tail.txt`, dan memori GPU proses PyTorch tahap A tidak bercampur dengan konteks TensorRT. Job membangun *engine* Vanilla, P2-Static, A3 gate, A3 heuristik, dan Stage 1 gate sebanyak `n_builds` (2) kali, karena latensi bervariasi antar-*build*.

Pada *build* pertama, `acc_pass()` menjalankan seluruh data uji. Untuk setiap citra, Vanilla, P2-Static, A3 paksa A, dan A3 paksa B dijalankan untuk akurasi, lalu A3 otomatis dijalankan dengan ambang B2 dari riwayat. Tiga paritas dicatat: selisih maksimum keluaran router terhadap rute paksa yang dipilih, selisih maksimum skor antar-mode, dan jumlah citra yang keputusan *engine*-nya berbeda dari perbandingan di host. Pada hasil final, keluaran router sama dengan rute paksa (selisih 0) dan tidak ada keputusan yang berbeda dari host, di keempat model.

`lat_pass()` mengukur latensi komponen pada 300 citra pertama, dengan 30 citra pemanasan dan 3 ulangan. Urutan varian dibalik di setiap ulangan agar efek urutan saling meniadakan. `e2e_pass()` mengukur latensi ujung-ke-ujung pada seluruh aliran uji dalam urutan berkas.

```
def timed(fn, im, sa):
    H["img"].copy_(im.float() / 255.0); H["sa"].fill_(sa); dsync()       # konversi di luar timer
    w0 = time.perf_counter()
    with ctx():
        u = fn()
    return (time.perf_counter() - w0) * 1e3, u
```

Untuk varian router, `fn()` mencakup hitung ambang B2 di host, salin citra dan ambang ke GPU, *engine*, salin `y`, `use_a`, dan skor ke host, lalu memasukkan skor ke riwayat. Konversi uint8 ke float dan penyalinan ke *buffer pinned* ada di luar *timer* untuk semua varian. Riwayat B2 dimulai ulang di setiap ulangan, sehingga keputusan setiap ulangan sama dengan pass akurasi; paritas ini dicatat sebagai `e2e_parity`. Dua router diukur: “B2 terkunci” (`np.quantile` atas `deque`) dan “B2 efisien” (`SortedWindowQuantile`). Urutan varian dibalik setiap (ulangan + citra) ganjil. Keluaran job adalah `job_res.json` (latensi per *build*, paritas, aktivasi, biaya host B2, ukuran *engine*) dan `job_dets.pkl` (deteksi dan keputusan per citra).

7.7 Analisis dan Kriteria

`analyze()` menggabungkan keluaran tahap A dan job. Langkah pertamanya menghitung latensi turunan. Dari `lat_pass` diperoleh latensi komposit $T_{router} = a \cdot T_A + (1 - a) \cdot T_B + T_\tau$ [Pers. (6)] dan fraksi campuran f [Pers. (7)], baik dengan biaya host B2 terkunci maupun efisien. Dari `e2e_pass` diperoleh latensi ujung-ke-ujung terukur dan fraksi f-nya. Semua f dipotong ke [0, 1].

Langkah kedua menyusun routing untuk *bootstrap*. Setiap routing adalah pasangan (`UnionBoot`, *mask*), dengan `U_AB` untuk A/B dan `U_VP` untuk P2-Static/Vanilla. *Mask* acak dibuat oleh `random_masks(n, f, n_draw, seed + offset)`, yang memilih tepat round($f \cdot n$) citra. *Offset seed* tetap per keluarga: +11 untuk acak pada aktivasi router, +13 untuk acak top-ρ, +17 untuk acak pada aktivasi heuristik, +19 untuk semua campuran router, dan +23 untuk campuran heuristik. Karena semua campuran router memakai *offset* +19, campuran untuk f yang berbeda hanya berbeda karena jumlah citra yang dipilih, bukan karena aliran bilangan acak yang berbeda. Rancangan ini mengurangi derau saat membandingkan varian H6.

Tabel 7.1. Uji di `bootstrap_test.csv` (keluarga uji yang sama untuk setiap model).

Label di kode	a	Pembanding	Arti
H2/H7 | router (B2) − acak (aktivasi sama)	router	5 acak, *offset* +11	Tesis H2 dan bagian H6
router (top-ρ) − acak (ρ)	router_top	3 acak, +13	Gate tanpa B2
H6 | router (B2) − campuran acak Vanilla/P2-Static (latensi sama)	router	5 campuran, f komposit naif	H6 Tahap 4 (kontinuitas)
router (B2) − Branch A (P2 selalu)	router	A	Biaya akurasi melewatkan P2
router (B2) − Branch B (P2 tidak pernah)	router	B	Manfaat P2 selektif
Branch A − P2-Static, Branch B − Vanilla	A, B	P2S, V	Kualitas rute vs model mandiri
gate − heuristik kepadatan (top-ρ), gate (B2) − heuristik kepadatan (B2)	router	heuristik	Gate vs pembanding tanpa label
heuristik (B2) − acak, heuristik (B2) − campuran	heur	acak, campuran	Mutu heuristik
router (B2) − kaskade bergate Vanilla → P2-Static	router	kaskade	Pembanding kaskade
[sensitivitas, pasca-hasil] H6 | router (B2 efisien) − campuran …	router	5 campuran, f komposit efisien	Sensitivitas
[4B, pasca-hasil] H6 | … (latensi ujung-ke-ujung)	router	5 campuran, f e2e naif	Sensitivitas
H6 utama | router (B2 efisien) − campuran acak V/P2S (latensi ujung-ke-ujung)	router	5 campuran, f e2e efisien	**Tesis H5**

Perhatikan bahwa *mask* router sama di semua uji ini. Keputusan B2 terkunci dan B2 efisien identik, sehingga perbedaan antara varian H6 hanya ada di pembandingnya, yaitu fraksi f yang diturunkan dari ukuran latensi yang berbeda.

Langkah berikutnya adalah *bootstrap* klaster untuk VisDrone (bila `cluster_by` diset), dengan ID video dari token pertama nama berkas. Hasilnya, `bootstrap_klaster.csv`, memuat CI per citra dan CI klaster berdampingan, rasio lebar keduanya, dan estimasi titik tanpa klaster terbesar. Setelah itu dihitung sensitivitas H6 per *build* (`h6_per_build.csv`), tabel COCO lengkap 12 metrik dengan latensi dan aktivasi (`coco_lengkap.csv`), perilaku skor untuk H3, keputusan B2 pada lima urutan aliran acak (`b2_urutan.csv`), dan pemeriksaan silang PyTorch FP32 (`cek_fp32.csv`).

H3 diuji sebagai korelasi Spearman parsial antara skor gate dan proporsi GT kecil, dengan jumlah GT sebagai kontrol, pada citra yang punya GT. Koreksi Holm atas empat model diterapkan saat `merge`, bukan di sini. Bagian `sanity` di `hasil.json` mencatat korelasi skor TensorRT vs PyTorch, kesepakatan keputusan job vs luring, kesepakatan keputusan FP16 vs FP32, dan jumlah nilai heuristik unik.

Kriteria per model ditulis di `hasil.json` bagian `kriteria`. Tabel 7.2 memetakan kuncinya ke kriteria tesis.

Tabel 7.2. Kunci `kriteria` di `hasil.json`.

Kunci	Isi	Kriteria tesis
`H1_mAP50`, `H1_latensi_e2e_efisien`	mAP50 dan latensi e2e router di antara Vanilla dan P2-Static	H1
`H1_latensi`, `H1_latensi_e2e`	Varian komposit dan e2e naif	sensitivitas
`H2_router_vs_acak`	Batas bawah CI mAP_small router − acak > 0	H2
`H5_rute_A_e2e`, `H5_rute_B_e2e`	Rute A − P2-Static dan rute B − Vanilla ≤ 0,15 ms (e2e)	H4
`H5_rute_A`, `H5_rute_B`	Sama, pada rute paksa `lat_pass`	sensitivitas
`H6_utama_status`	“unggul (CI > 0)”, “tidak kalah (CI memuat 0)”, atau “kalah (CI < 0)”	H5
`H6_status`, `H6_e2e_status`, `H6_sensitivitas_B2_efisien`	Varian H6 lain	sensitivitas
`H7_aktivasi`, `H7_gain`	|aktivasi − ρ| ≤ 0,05 dan H2 lolos	H6

7.8 Menjalankan, Mengarsip, dan Menggabung

`run_model()` menjalankan ketiga tahap dengan pola *resume*. Bila `stageA.pkl` sudah ada, tahap A dilewati. Bila `job_res.json` dan `job_dets.pkl` sudah ada, job dilewati. Dengan opsi `--reuse`, berkas *intermediate* dari sesi lain disalin lebih dulu, sehingga analisis dapat diulang di CPU tanpa GPU. Penanda `DONE` ditulis bila analisis selesai. `zip_model()` membuat dua arsip: hasil (tanpa ONNX, *engine*, `.pkl`, dan GT) dan *intermediate* (`stageA.pkl`, `job_res.json`, `job_dets.pkl`, `gt_test.json`).

`merge()` mengumpulkan folder hasil dari arsip atau dataset Kaggle. Bila sebuah model punya beberapa salinan, salinan yang dipilih adalah yang punya `h6_per_build.csv` dan paling baru. Fungsi ini lalu menyusun tabel kriteria (`ringkasan_tab.csv`), tabel COCO semua model (`ringkasan_full.csv`), H3 dengan koreksi Holm atas model (`ringkasan_h3.csv`; lolos bila ρ > 0 dan p Holm < 0,05), latensi gabungan (`ringkasan_latp.csv`), *pooling* tiga *seed* KITTI (`ringkasan_pool.csv`), dan grafik Pareto mAP_small–latensi (`pareto.png`). Notebook menjalankan `merge` dua kali, sekali untuk pembanding `last.pt` dan sekali untuk pembanding `best.pt`, supaya hasil sensitivitas tidak tercampur ke *pooling* utama.

8 Analisis Pelengkap: ablation.py, ckcheck.py, profile.py, latency_l2.py, a12.py

8.1 Ablasi Pasca-Pelatihan

`ablation.py` menjawab pertanyaan mengapa setiap pilihan desain gate dan anggaran masuk sistem final. Semua varian dijalankan di atas detektor final yang dibekukan, dan pilihan dinilai di **val**. Uji hanya dipakai untuk seri anggaran (B), yang mengukur ketahanan aktivasi pada aliran nyata, bukan untuk seleksi.

Kelas `Split` menyimpan *cache* satu split: fitur, deteksi A/B, objek `Router`, dan target per citra. Metode `gain()` mendefinisikan ukuran yang dipakai di seluruh ablasi, yaitu metrik routing dikurangi rata-rata metrik 20 routing acak dengan aktivasi yang sama.

```
def gain(self, use_a, n_random, seed):
    m = self.R.metrics(use_a)
    base = [self.R.metrics(r) for r in random_masks(self.n, float(np.mean(use_a)), n_random, seed)]
    return {f"gain_{k}": m[k] - float(np.mean([b[k] for b in base])) for k in METRICS}
```

`run_ablation()` menjalankan lima seri. G1 membandingkan target gate: `soft_small` (final), `soft_all`, `net25_all`, selisih loss per citra (label 1 bila loss A < loss B), dan selisih loss dikurangi median (ala *offset* DynamicDet), ditambah dua plafon ber-GT di val (oracle Δ soft_small dan jumlah GT kecil). Selisih loss dihitung oleh `per_image_losses()`, yang menjalankan `BranchLoss` stok per citra dengan *batch* 1. G3 membandingkan subset fitur lewat `feature_columns()`: empat peta GAP+GMP (final), tanpa `p2_backbone`, hanya `p3_neck`, hanya GAP, dan hanya GMP. G4 membandingkan C regresi logistik dari 10^{-4} sampai 1 dan `HistGradientBoostingClassifier`. G5 mengukur biaya label dengan subset train berlabel 50, 100, 250, 500, 1.000, 2.500, dan semua, dengan lima subset acak per ukuran. Seri B membandingkan ambang val tetap, B2 dengan W = 50, 200, dan 500, serta top-ρ transduktif pada aliran uji. Seri G1 sampai G5 memakai pemilihan top-ρ (ρ = 0,5), sehingga aktivasinya sama dan perbandingannya hanya soal peringkat.

`boot_ablation()` menguji 15 pasangan yang didefinisikan di `PAIRS`, misalnya `G1 soft_small − selisih loss` dan `B B2 window 200 − threshold val tetap`. Untuk setiap pasangan, statistiknya adalah selisih *gain* kedua varian, dengan *bootstrap* berpasangan atas citra. Ada satu detail rancangan di sini: himpunan routing acak dipakai bersama untuk semua varian yang aktivasinya sama (`rand_of[k]`, *seed* `rand_seed + k`). Untuk pasangan top-ρ, aktivasinya selalu sama, sehingga suku acaknya saling meniadakan dan selisih *gain* sama dengan selisih metrik persis. Uji

`test_boot_ablation_point_exact` memeriksa sifat ini. Koreksi Holm diterapkan atas seluruh keluarga pasangan per metrik. Untuk VisDrone, versi klaster per video juga dihitung dengan estimasi titik yang sama. Komentar kode menyatakan bahwa pasangan dipilih setelah estimasi titik ablasi *seed* 42 terlihat, sehingga uji ini dilaporkan sebagai konfirmasi ketidakpastian, bukan seleksi desain.

8.2 Cek Checkpoint

`ckcheck.py` ada karena satu ketidakcocokan. Loss pelatihan adalah L_A + 0,7 · L_B, tetapi `best.pt` Ultralytics dipilih dari *fitness* cabang A saja, yaitu 0,1 · mAP50 + 0,9 · mAP50-95. *Epoch* terbaik untuk A belum tentu terbaik untuk B, dan memilih *epoch* dari A dapat membesarkan jarak A − B, yang justru menjadi target gate. Karena itu `last.pt` dikunci sebelum pelatihan, dan skrip ini hanya melaporkan selisihnya.

`best_epoch_from_results()` membaca `results.csv` dan mencari *epoch* dengan *fitness* maksimum; bila ada seri, *epoch* terakhir yang diambil, sesuai perilaku `best.pt`. `evaluate_pair()` menjalankan *cache* val untuk kedua *checkpoint*, lalu menghitung tiga hal. Yang pertama adalah metrik titik keempat sumber (A dan B, `last` dan `best`). Yang kedua adalah tiga uji *bootstrap* berpasangan: A_last − A_best, B_last − B_best, dan selisih jarak (A_last − B_last) − (A_best − B_best). Untuk uji ini, satu `UnionBoot` dibuat per cabang dengan `last` sebagai sumber pertama dan `best` sebagai sumber kedua, sehingga *mask* "semua A" dan "semua B" mewakili kedua *checkpoint* pada indeks *resample* yang sama. Yang ketiga adalah plafon oracle per *checkpoint*. `verdict()` menulis ringkasan teks, dan tidak mengubah pilihan *checkpoint*.

8.3 Profil Latensi per Segmen

`profile.py` menjawab pertanyaan diagnostik: dari mana *overhead* rute B (0,10–0,13 ms di atas Vanilla) dan keputusan host berasal. Setiap pemanggilan dibagi menjadi segmen dengan empat *CUDA event* di *stream* yang sama.

```
def __call__(self, op):
    marks = []
    if self.cuda:
        def mark():
            self.ev[len(marks)].record(self.s); marks.append(None)
        torch.cuda.synchronize()
        w0 = time.perf_counter()
        with torch.cuda.stream(self.s):
            op(mark, self.s)
        self.s.synchronize()
        wall = (time.perf_counter() - w0) * 1e3
        seg = [self.ev[i].elapsed_time(self.ev[i + 1]) for i in range(3)]
        gpu = self.ev[0].elapsed_time(self.ev[3])
    # ...
    return dict(h2d=seg[0], exec=seg[1], d2h=seg[2], gpu_total=gpu, wall=wall, host=wall - gpu)
```

Setiap operasi memanggil `mark()` empat kali: di awal, setelah salin citra ke GPU, setelah *engine*, dan setelah salin keluaran ke host. Segmen `host` adalah waktu dinding dikurangi total waktu GPU, yaitu biaya peluncuran, sinkronisasi, dan Python. Varian yang diukur adalah Vanilla, P2-Static, A3 rute A, A3 rute B, A3 rute B yang hanya menyalin 8.400 *anchor*, rantai dua *engine* tanpa `If` (Stage 1 lalu Stage 2A atau 2B, dengan masukan Stage 2 diikat langsung ke *buffer* keluaran Stage 1 tanpa salinan), dan Stage 1 saja. `derive()` menghitung enam selisih diagnostik (`DIFFS`), misalnya "rute B salin semua − salin 8.400", yang mengisolasi biaya *padding* di salinan. *Clock* GPU dicatat sebelum dan sesudah pengukuran, karena variasi *clock* dapat menyamar sebagai selisih latensi.

8.4 Latensi L2

`latency_l2.py` mengukur batas sistem yang lebih lebar daripada L1. L1 (dasar H5 tesis) mencakup ambang B2, salin citra FP32 640 × 640, *engine*, dan salin keluaran mentah. L2 mencakup ambang B2,

salin citra uint8 ukuran asli, *letterbox* dan normalisasi di GPU, *engine*, keputusan, dekode dan NMS di GPU, lalu salin kotak ke host. Pembacaan berkas (`cv2.imread`) tetap di luar *timer*.

`GpuLetterbox` meniru jalur val Ultralytics 8.3.240: ubah ukuran sisi panjang dengan interpolasi bilinear (`align_corners=False`) lalu dibulatkan ke 0–255, *padding* 114 di tengah, BGR ke RGB, dan bagi 255. Hasilnya tidak identik bit per bit dengan `cv2.resize`, dan uji `test_letterbox_matches_ultralytics` mengizinkan selisih paling besar satu tingkat piksel. Karena itu, kesepakatan keputusan router L2 terhadap L1 dilaporkan, dan aturan bacanya ditulis sebelum angka L2 ada: bila kesepakatan di bawah 0,99, H5 pada L2 tidak dilaporkan sebagai uji.

`nms_gpu()` memakai `torchvision.ops.batched_nms` dengan dua mode. Mode satu label (conf 0,25) adalah perilaku `predict` Ultralytics dan dianggap mode *deployment*. Mode *multi-label* (conf 0,001) sama dengan protokol val dan menjadi batas atas biaya NMS. Uji `test_nms_matches_eval_and_predict` membandingkan keduanya dengan NMS Ultralytics. Untuk router, ada dua varian. Varian utama tidak menunggu keputusan di tengah: NMS selalu dijalankan atas 34.000 *anchor*, karena *anchor padding* berskor 0 tidak lolos conf, sehingga host cukup satu sinkronisasi seperti model statis. Varian naif menunggu keputusan dulu, lalu memilih jumlah *anchor*. Kedua varian harus memberi keputusan yang sama, dan `l2_pass()` memeriksanya dengan `assert`. `l1_decisions()` menghitung keputusan acuan dengan *letterbox* CPU untuk kesepakatan L2–L1.

8.5 A12: Engine A3T

`a12.py` menguji satu rancangan lanjutan untuk menghapus biaya *padding*. Dalam tata letak (1, 6, N), 8.400 *anchor* rute B membentuk enam potongan yang tidak bersebelahan di memori, sehingga menyalin “hanya 8.400 *anchor*” tetap tidak murah dan hasilnya tidak konsisten antar-sesi. `transpose_output()` menambahkan satu `Transpose` setelah `If` pada graf ONNX yang sama, sehingga keluarannya menjadi (1, N, 6) dan *anchor* rute B menjadi awalan bersebelahan.

```
def a3t_two_stage():
    """Router: awalan + keputusan dulu; sisa anchor hanya bila rute A. Kembali: keputusan
(bool)."""
    launch(-1.0)
    ty_pre.copy_(t.buf["y"][:, :n_b], non_blocking=True); tu.copy_(t.buf["use_a"],
non_blocking=True)
    ts.copy_(t.buf["score"], non_blocking=True); sync()
    use = bool(float(tu[0]) > 0.5)
    if use:
        ty_rest.copy_(t.buf["y"][:, n_b:], non_blocking=True); sync()
    return use
```

Router A3T “dua tahap” selalu menyalin awalan 8.400 *anchor* beserta keputusan dan skor, lalu menyalin 25.600 *anchor* sisanya hanya bila rutenya A. Rute B menghemat salinan, tetapi rute A membayar satu salinan dan satu sinkronisasi tambahan. Karena nilai keluaran sama dan hanya tata letaknya berubah, akurasi tidak berubah, dan `parity()` memeriksa bahwa `y_t` sama dengan transpos `y` untuk kedua rute, bahwa *padding* A3T nol, dan bahwa skornya sama. Pembandingnya wajib router A3 asli di sesi dan aliran yang sama, karena variasi latensi antar-sesi (sekitar 0,05 ms) sebesar efek yang dicari.

9 Lingkungan, Provenans, dan Reproduksi: env.py, repro.py, compat.py

9.1 Penjaga Ultralytics Stok dan Provenans

Reproduksi hanya bermakna bila kode yang berjalan sama. Repo tesis punya folder `ultralytics/` berisi *fork* lama yang masih memuat router lama. Bila skrip dijalankan dari root repo, Python akan mengimpor *fork* itu, bukan paket PyPI, tanpa pesan apa pun. `require_stock_ultralytics()` mencegah hal ini.

```
def require_stock_ultralytics():
    import ultralytics
    pkg = Path(ultralytics.__file__).resolve().parent
    problems = []
    if ultralytics.__version__ != ULTRALYTICS_VERSION:
        problems.append(f"versi {ultralytics.__version__} != {ULTRALYTICS_VERSION}")
    if (pkg / "model_yolov8_router.py").exists():
        problems.append(f"{pkg} adalah fork router lama (ada model_yolov8_router.py)")
    for rel, want in STOCK_SHA256.items():
        p = pkg / rel
        if not p.is_file() or sha256_file(p) != want:
            problems.append(f"{rel} berbeda dari wheel stok")
    if problems:
        raise RuntimeError(...)
```

Pemeriksaannya tiga lapis: nomor versi harus 8.3.240, berkas penanda *fork* (`model_yolov8_router.py`) tidak boleh ada, dan sha256 delapan berkas inti harus sama dengan *wheel* resmi PyPI. Kedelapan berkas itu adalah `nn/tasks.py`, `nn/modules/head.py`, `nn/modules/block.py`, `utils/loss.py`, `utils/tal.py`, `engine/trainer.py`, `engine/validator.py`, dan `models/yolo/detect/train.py`, yaitu semua berkas yang menentukan graf, loss, *assigner*, pelatihan, dan validasi. Fungsi ini dipanggil oleh `build_trainer()`, `provenance()`, suite, dan uji `test_stock_ultralytics`. Pada uji lokal untuk dokumen ini, pemeriksaan ini lolos dengan paket PyPI 8.3.240.

`provenance()` menulis catatan lingkungan setiap run: waktu, `argv`, commit dan *branch* `router_final` beserta daftar berkas *dirty* (`git status --porcelain` untuk folder ini), lokasi paket Ultralytics, versi pustaka (Python, torch, numpy, CUDA, cuDNN, ultralytics, scikit-learn, pycocotools, onnx, onnxruntime, TensorRT, scipy, OpenCV), GPU dan *driver*, beberapa variabel lingkungan yang memengaruhi determinisme, dan sha256 berkas masukan. Daftar berkas *dirty* inilah yang dipakai verifikasi B4 dan B8 untuk memastikan run tertentu berasal dari commit bersih. `load_config()` membaca YAML dan mengubah path model menjadi path absolut relatif terhadap `configs/`.

9.2 Sidik Jari Bobot

sha256 berkas `.pt` tidak cocok untuk uji reproduksi, karena berkas *checkpoint* memuat metadata yang selalu berbeda (tanggal, nama run, `argv`). `state_hash()` menghitung sha256 atas tensor saja.

```
def state_hash(obj):
    sd = obj.state_dict() if hasattr(obj, "state_dict") else obj
    h = hashlib.sha256()
    for k in sorted(sd):
        v = sd[k]
        if not torch.is_tensor(v):
            continue
        t = v.detach().cpu().contiguous()
        h.update(k.encode()); h.update(str(t.dtype).encode());
h.update(str(tuple(t.shape)).encode())
        h.update(t.reshape(-1).view(torch.uint8).numpy().tobytes() if t.numel() else b"")
    return h.hexdigest()
```

Nama, tipe, bentuk, dan byte setiap tensor ikut di-*hash*, urut nama. Dua model dengan hash sama berarti bobot dan *buffer*-nya (termasuk statistik BatchNorm) identik bit per bit. `max_abs_diff()`

dipakai untuk diagnosis bila hash berbeda, dengan mencari tensor mana yang berbeda dan seberapa jauh.

Status jaminan reproduksi perlu dibaca dengan teliti. README menyatakan pelatihan CPU dengan *seed* sama sudah diuji identik bit per bit, dan pada saat README ditulis uji GPU belum dijalankan. Setelah itu, uji GPU dijalankan di Kaggle T4 dengan `check_reproducibility.py` (dua run, masing-masing dua *epoch*). Untuk VisDrone *seed* 42, log dan arsipnya tersimpan: `state_hash` kedua run identik (`c7a86dd7…`), dan *epoch* pertama pelatihan final identik dengan *epoch* pertama uji reproduksi di 19 kolom `results.csv`. Untuk tiga *seed* KITTI, hasil lolos tercatat dari keluaran notebook, tetapi lognya tidak diarsip. Jaminan ini berlaku untuk lingkungan yang sama (GPU, *driver*, versi torch/CUDA, *batch*, dan jumlah *worker*); di lingkungan lain, bobot dapat berbeda.

9.3 Pemetaan Bobot M0

`compat.py` hanya dipakai untuk uji kesetaraan graf. Fungsi `map_m0_state_dict()` memetakan kunci *state_dict* model lama ke graf final.

```
if i <= 15:
    out[k] = v
elif i == 16:
    if rest.startswith("c2f_p2."):
        out[f"model.18.{rest[len('c2f_p2.'):]}"] = v
    elif rest.startswith(DROPPED_ROUTER_PREFIXES):
        dropped.append(k)
    else:
        raise KeyError(f"kunci router tak dikenal: {k}")
else:
    out[f"model.{i + 2}.{rest}"] = v
```

Di model lama, layer 16 adalah modul router yang berisi `c2f_p2` (jalur P2) dan komponen keputusan. Di graf final, jalur P2 dipecah menjadi tiga layer stok (16 *upsample*, 17 *concat*, 18 C2f), dan hanya layer 18 yang berparameter. Karena itu `c2f_p2.*` dipetakan ke `model.18.*`, komponen router (`squeeze_*`, `classifier`, `proxy_*`, `tau_param`, `_calib_*`) dibuang, dan semua layer setelahnya bergeser dua indeks. Alias `detect_A.*` dan `detect_B.*` dilewati. Kunci layer 16 yang tidak dikenali menimbulkan galat, sehingga tidak ada bobot router yang hilang diam-diam. Pada M0 *seed* 42, pemetaan ini membuang 30 kunci router, dan graf final dengan bobot hasil pemetaan memberi fitur *trunk*, keluaran cabang A dan B, serta item loss yang identik bit per bit dengan model lama.

10 Skrip Baris Perintah

Setiap skrip di `scripts/` adalah lapisan tipis di atas pustaka `rfinal/`: membaca argumen dan konfigurasi, memanggil fungsi pustaka, menulis keluaran, dan menulis `provenance.json`. Skrip mengimpor `_path.py` lebih dulu, yang menambahkan folder `router_final/` ke `sys.path`, sehingga skrip dapat dijalankan dari folder mana pun. Namun direktori kerja tetap tidak boleh root repo, karena di sana folder `ultralytics/` milik *fork* lama akan menutupi paket PyPI. Tabel 10.1 merangkum semua skrip.

Tabel 10.1. Skrip CLI, argumen penting, dan keluaran.

Skrip	Argumen penting	Keluaran dan catatan
`train.py`	`--config`, `--seed`, `--model {dualbranch, vanilla, p2static}`, `--override`, `--strict-determinism`	Folder run Ultralytics: `weights/last.pt`, `results.csv`, `args.yaml`, `branch_b_metrics.csv`, `provenance.json`, `state_hash.json`, `weights_sha256.json`. *Pembanding dilatih dengan resep yang sama.*
`check_reproducibility.py`	`--config`, `--seed`, `--override epochs=2`, atau `--compare RUN1 RUN2`	Dua folder run `repro_*`, kode keluar 0/1. *Membandingkan `state_hash.json`.*
`check_checkpoint.py`	`--config`, `--run`, `--out`, `--n-boot`,	`titik.csv`, `uji.csv`, `plafon_oracle.csv`,

Skrip	Argumen penting	Keluaran dan catatan
	`--seed 20261021`	`lintasan_epoch.csv`, `ringkasan.txt`. *Hanya val; laporan, bukan seleksi.*
`fit_gate.py`	`--config`, `--weights`, `--out`	`gate.json`, `target_train.csv`, `skor_val.csv`, `gt_train.json`. *cuDNN deterministik.*
`export.py`	`--config`, `--weights`, `--gate`, `--out`, `--precision`, `--trt`	ONNX `s1/s2a/s2b/a3` FP32 dan FP16, `a3_fp16.engine`, `deploy.json`. *Verifikasi pada 4 citra val.*
`evaluate.py`	`--backend {pytorch, ort, trt}`, `--engine`, `--split`, `--n-random`	`metrik.csv`, `keputusan_per_gambar.csv`. *Pemeriksaan cepat.*
`check_fast_nms.py`	`--k 2000 5000 10000`, `--precisions fp32 fp16`, `--n`, `--assert-config`	`kandidat.csv`, `ringkasan.csv`, `selisih_metrik.csv`. *Kode keluar 1 bila K di config tidak identik.*
`suite.py run`	`--config`, `--seed`, `--weights`, `--gate`, `--vanilla`, `--p2static`, `--out`, `--zip`, `--reuse`, `--override`	Per model: `hasil.json`, `bootstrap_test.csv`, `coco_lengkap.csv`, `latensi*.csv`, `per_gambar_test.csv`, dan lainnya. *Protokol terkunci.*
`suite.py merge`	`--out`, `--models`, `--glob`	`ringkasan_tab/full/h3/latp/pool.csv`, `pareto.png`. *Tanpa GPU.*
`ablate.py`	`--config`, `--weights`, `--out`, `--reuse-cache`, `--no-boot`, `--n-random-boot`	`ablasi.csv`, `ablasi_uji.csv`, `ablasi_uji_klaster.csv`, `cache_ablasi.pkl`. *`--reuse-cache` tanpa GPU.*
`profile_latency.py`	`--no-l2`, `--l2-only`, `--l2-conf 0.25 0.001`, `--n-builds`, `--n-images`	`segmen.csv`, `selisih.csv`, `paritas_rantai.csv`, `latensi_l2.csv`, `keputusan_l2.csv`, `l2.json`, `clock_*.txt`. *Diagnostik.*
`h6_latency.py`	`--suite-dir`, `--l2-json`, `--ref`	`h6_latensi.csv`, `ringkasan.txt`. *Tanpa GPU.*
`h6_latency_pool.py`	`--dirs`, `--out`	*Pooling* H6 L1/L2 tiga *seed* KITTI. *Tanpa GPU.*
`sweep_rho.py run/pool`	`--suite-dir`, `--ablation-cache`, `--grid`, `--ref`, `--fast`	`sweep_test.csv`, `sweep_val.csv`, `sweep_latensi.csv`, `sweep_pool.csv`. *ρ = 0,5 tetap terkunci.*
`cluster_boot.py run/pool`	`--cluster {csv, video_prefix}`, `--map-csv`, `--ref`, `--ref-cluster`	`bootstrap_klaster_a2.csv`, `bootstrap_klaster_a2_pool.csv`. *A2, tanpa GPU.*
`a11_bandingkan_profil.py`	`--dirs LABEL=FOLDER`, `--sesi`, `--out`	`a11_run.csv`, `a11_antar_sesi.csv`, `a11_antar_semua.csv`. *Aturan 0,02 ms.*
`a12_engine_tanpa_padding.py`	sama dengan `profile_latency.py`, + `--suite-dir`, `--ref`	`latensi_a12.csv`, `a12.json`, `keputusan_l2.csv`, `h6/`. *Kontrol di sesi yang sama.*

Beberapa skrip punya logika sendiri yang perlu diketahui. `check_reproducibility.py` menjalankan `train.py` dua kali sebagai proses terpisah dengan resep yang sama, lalu membandingkan `state_hash.json`. Bila berbeda, skrip mencetak lima tensor dengan selisih terbesar dan bagian `provenance.json` yang berbeda (versi, perangkat, variabel lingkungan), supaya penyebabnya dapat dilacak. Kode keluarnya 0 bila identik dan 1 bila berbeda. Di notebook, kegagalan uji ini dicetak sebagai peringatan dan tidak menghentikan notebook.

`check_fast_nms.py` membandingkan `to_dets(fast_nms_k = K)` dengan `to_dets()` penuh byte per byte, per citra val, untuk cabang A dan B, FP32 dan FP16, setiap `max_det`, dan setiap K. Varian FP16 memakai `model.half()` untuk meniru pola skor kembar keluaran TensorRT FP16, karena celah satu-satunya NMS cepat adalah skor kembar. Bila ada yang tidak identik, skrip menghitung selisih metrik COCO yang ditimbulkannya.

`h6_latency.py`, `sweep_rho.py`, dan `cluster_boot.py` bekerja dari *intermediate* suite tanpa GPU. Ketiganya memakai deteksi dan keputusan dari pass akurasi suite, *offset seed* undian yang sama, dan indeks *resample* yang sama. Karena itu ketiganya punya pemeriksaan paritas yang sama: baris yang sesuai dengan kondisi utama suite harus mereproduksi `bootstrap_test.csv` suite dengan selisih nol sebelum baris lain boleh dibaca. `h6_latency.py` hanya mengganti fraksi campuran f dengan f dari latensi lain (L2 atau A12), dan menolak menafsirkan H5 L2 bila kesepakatan keputusan L2–L1 di bawah 0,99. `sweep_rho.py` menurunkan f(ρ) dari model latensi linear T(a) = a · T_A + (1 − a) · T_B + h, dengan h dikalibrasi dari titik terukur. Model ini mereproduksi f utama di ρ = 0,5 secara persis; titik ρ lain adalah perkiraan, dan ini ditulis di *docstring* skrip. `cluster_boot.py` membentuk klaster KITTI dari kolom `rekaman` di `kitti_peta_rekaman.csv` (pemetaan *devkit* KITTI). Kode klasternya divalidasi dengan menjalankannya pada VisDrone dengan klaster video, yang harus mereproduksi `bootstrap_klaster.csv` suite.

`a11_bandingkan_profil.py` menerapkan aturan keputusan A11 yang ditulis sebelum dijalankan: bila rincian rute B − Vanilla berbeda lebih dari 0,02 ms per segmen antar-sesi, rentangnya dilaporkan; bila tidak, ditulis “direplikasi pada dua sesi”. Skrip juga memperingatkan bila dua label ternyata menunjuk ke run yang sama.

11 Pengujian dan Alat Bantu

11.1 Uji Unit dan Uji Kesetaraan

Folder `tests/` berisi 23 fungsi uji dalam 13 berkas. Uji *letterbox* diparametrisasi dengan enam ukuran citra, sehingga `pytest` mengumpulkan 28 kasus uji. `conftest.py` hanya menambahkan folder `router_final/` ke `sys.path` supaya `pytest` dapat mengimpor `rfinal`. Semua uji berjalan di CPU tanpa GPU dan tanpa dataset; data yang dibutuhkan dibuat sintetis, atau dibaca dari berkas acuan bila variabel lingkungannya diisi. Notebook menjalankan semua uji di sel pertama sebelum langkah apa pun, sehingga lingkungan Kaggle yang rusak terdeteksi dalam kurang dari satu menit. Tabel 11.1 merangkum isi uji dan hasil uji ulang untuk dokumen ini.

Tabel 11.1. Uji di `tests/` dan hasilnya pada uji ulang 2 November 2026 (CPU).

Berkas	**Yang diperiksa**	**Hasil**
`test_model.py` (5)	Ultralytics stok; *stride* A dan B; tidak ada kunci router atau alias di `state_dict`; *trunk* + cabang = *forward* penuh (`torch.equal`), bentuk 34.000/8.400/960; loss 6 item dan semua parameter menerima gradien; inisialisasi deterministik	lolos
`test_gate.py` (1)	Lipatan scaler + logreg; simpan dan muat `gate.json` bit-identik; Gemm FP32 ≈ numpy float64	lolos
`test_budget.py` (2)	B2 = salinan apa adanya Tahap 4; versi *streaming* = versi *batch*; B2 efisien bit-identik dengan `np.quantile` untuk ρ 0,25–0,7 dengan skor kembar	lolos
`test_export.py` (1)	Graf A3 ONNX = PyTorch untuk tiga mode dan dua arah ambang ($< 10^{-4}$)	lolos
`test_cocomix.py` (1)	`Router`, `UnionBoot`, `FastUnionBoot` = COCOeval langsung (selisih $< 10^{-12}$) pada data sintetis; `UnionBoot` = `FastUnionBoot` pada *resample* berulang	lolos
`test_fast_nms.py` (2)	NMS cepat identik dengan NMS penuh dan kedua jalur benar-benar terpakai; sifat awalan `d[:max_det]`	lolos
`test_ablation_boot.py` (2)	Estimasi titik `boot_ablation` = selisih *gain*	lolos

Berkas	Yang diperiksa	Hasil
	langsung; versi klaster punya estimasi titik sama	
`test_ckcheck.py` (1)	Pemilihan *epoch* `best.pt` dari `results.csv`, seri ke *epoch* terakhir	lolos
`test_latency_l2.py` (3 fungsi, 8 kasus)	*Letterbox* GPU vs Ultralytics (≤ 1 tingkat piksel); NMS GPU vs NMS val dan `predict`; NMS rute B mengabaikan *anchor padding*	lolos
`test_profile.py` (1)	`Timer` (CPU), `summarize`, `derive`	lolos
`test_a12.py` (1)	`transpose_output` pada graf `If` sintetis = transpos keluaran asli	lolos
`test_m0_equivalence.py` (2)	Graf final dengan bobot M0 = model lama (inferensi dan item loss)	dilewati: butuh `RFINAL_M0_REF`
`test_suite_replay.py` (1)	`analyze` mereproduksi Tahap 4/4B per *resample* pada *intermediate* Kaggle	dilewati: butuh `RFINAL_SUITE_REF`

Totalnya 25 kasus lolos dan 3 dilewati, dalam sekitar 23 detik. Angka dalam kurung di kolom pertama adalah jumlah fungsi uji. Uji yang dilewati tidak gagal; ketiganya memang dirancang hanya berjalan bila berkas acuan dari luar diberikan. Berkas acuan M0 hanya dapat dibuat dengan *fork* lama, dan *intermediate* Tahap 4 ada di arsip hasil Kaggle. Menurut README, kedua uji itu sudah dijalankan dan lolos pada saat port dibuat; uji ulang untuk dokumen ini tidak dapat mengonfirmasinya karena berkas acuannya tidak ada di salinan kode.

Ada dua hal yang membuat uji ini bermakna, bukan sekadar formalitas. Pertama, banyak uji membandingkan dengan **acuan independen**, bukan dengan kode itu sendiri: COCOeval asli untuk `CocoMix`, NMS Ultralytics untuk NMS cepat dan NMS GPU, `np.quantile` untuk B2 efisien, scikit-learn untuk lipatan gate, ONNX Runtime untuk graf A3, dan salinan kode Tahap 4 untuk B2. Kedua, data sintetis dibuat tidak trivial. Misalnya, `test_cocomix.py` menuntut mAP_small di atas 0,01 dan mAP50 di atas 0,1 pada data sintetisnya, supaya uji kesamaan tidak lolos hanya karena semua metrik bernilai nol. `test_fast_nms.py` juga memeriksa bahwa jalur cepat dan jalur penuh sama-sama terpakai, supaya uji tidak lolos hanya karena selalu jatuh ke NMS penuh.

11.2 Rantai Validasi terhadap M0

Lima alat di `tools/` membentuk rantai yang membuktikan bahwa `router_final` menghitung hal yang sama dengan kode versi M0. Tiga alat pertama harus dijalankan di dua lingkungan berbeda, karena model M0 hanya dapat dimuat dengan *fork* lama.

`m0_state_dict.py` dijalankan dengan *fork* lama untuk mengekstrak `state_dict` M0 menjadi berkas tensor biasa. `m0_reference.py`, juga dengan *fork* lama, membuat berkas acuan berisi `state_dict` M0, keluaran cabang A dan B serta fitur *trunk* untuk beberapa citra uji, dan keluaran `Detect` mode *training* beserta item loss lama untuk satu *batch*. Di lingkungan stok, `convert_m0.py` memetakan `state_dict` itu ke checkpoint `DualBranchDetector` lewat `compat.py`, dan `test_m0_equivalence.py` membandingkan graf final dengan berkas acuan. Hasilnya, menurut README, adalah identik bit per bit untuk fitur, keluaran, dan item loss. `compare_suite_runs.py` membandingkan dua folder hasil suite, misalnya Tahap 4B di *fork* lama pada bobot M0 dan `rfinal.suite` pada bobot M0 yang dipetakan. *Cache*, deteksi, skor, keputusan, dan semua uji *bootstrap* harus identik; hanya kolom latensi yang boleh berbeda.

Perlu diingat bahwa bobot M0 dipakai **hanya** untuk validasi kode. Model final dilatih ulang dengan resep final, dan semua angka tesis berasal dari model final.

11.3 Run Ulang dari Commit Bersih

`b8_sweep_rho_bersih.py` menjawab temuan verifikasi B4: sapuan ρ KITTI awalnya dijalankan dari commit dengan `sweep_rho.py` yang belum di-commit, sehingga provenansnya *dirty*. Alat ini membuat `git worktree` terpisah pada commit bersih (*checkout* segar, sehingga tidak ada perubahan CRLF atau berkas tak terlacak), memastikan `git status --porcelain` kosong, menjalankan ulang sapuan dengan masukan dari arsip hasil dan argumen yang sama dengan run asli, lalu membandingkan keluarannya dengan `sweep_rho_hasil` secara byte dan numerik. Alat ini juga memeriksa provenans (`dirty_files = []`) dan paritas ρ = 0,5 terhadap suite. Prosesnya tanpa GPU, tetapi membutuhkan `fork` multiproses sehingga harus dijalankan di Linux, WSL, atau Kaggle.

12 Notebook Kaggle

`notebooks/kaggle_router_final.ipynb` menjalankan pipeline di Kaggle Tesla T4, satu *seed* per sesi karena batas sesi 12 jam. Notebook tidak berisi logika eksperimen sendiri; setiap sel memanggil skrip di `scripts/`. Tabel 12.1 memetakan selnya. Kolom pertama memuat indeks sel dan, dalam kurung, nomor bagian yang tertulis di komentar sel; teks di bawah memakai nomor bagian.

Tabel 12.1. Sel notebook.

Sel	Isi	Langkah `STEPS`
0 (md)	Petunjuk sesi, *resume*, NMS cepat, log	—
1 (0)	Kode dipin: `CODE_COMMIT`, `SEED`, `CONFIG`, `SESSION`, `STEPS`, `RESUME_FROM`, `BASELINES`; `git checkout`; instal Ultralytics 8.3.240 dan pustaka *deployment*; fungsi `run_logged`	—
2 (1)	`pytest tests` (CPU, < 1 menit)	selalu
3 (2)	Uji reproduksi GPU (2 × 2 *epoch*)	`train` dan `RUN_REPRO_CHECK`
4 (3b)	Latih Vanilla dan P2-Static, lalu arsipkan	`baselines`
5 (3)	Pelatihan 100 *epoch* dan cek *checkpoint*, atau salin arsip dari `RESUME_FROM`	`train`
6 (4)	`fit_gate.py`	`gate`
7 (5)	`export.py --trt`	`export`
8 (6)	`evaluate.py` TensorRT, dan opsional PyTorch FP32	`eval`, `eval_fp32`
9 (6b)	`check_fast_nms.py --assert-config`	`nms_check`
10 (7)	`suite.py run` (pembanding `last.pt`)	`suite`
11 (7b)	`suite.py run` dengan pembanding `best.pt` (sensitivitas)	`sens_best`
12 (7c)	`profile_latency.py` (segmen dan/atau L2), lalu `h6_latency.py`	`profile`
13 (7d)	`a12_engine_tanpa_padding.py`	`a12`
14 (8)	`ablate.py`	`ablation`
15 (9)	Arsip folder run tanpa *engine*, ONNX, dan `epoch*.pt`	selalu
16–17 (10)	`suite.py merge` dua kali: pembanding `last.pt` dan `best.pt`	sesi gabungan

Bagian 0 (sel 1) adalah satu-satunya tempat pengguna mengubah sesuatu. `CODE_COMMIT` wajib berupa *hash* commit, bukan nama *branch*, dan notebook berhenti bila masih berisi teks pengganti. Dengan begitu, kode yang berjalan dapat dilacak tepat, karena nama *branch* dapat bergeser. Direktori kerja dipindah ke `router_final/`, bukan root repo, dengan alasan yang sama seperti penjaga Ultralytics stok. Ultralytics dipasang dengan `--no-deps` supaya versi torch bawaan Kaggle tidak ikut diganti.

Variabel `SESSION` memilih preset `STEPS`. `SESSION = 1` menjalankan pelatihan, cek *checkpoint*, gate, ekspor, dan evaluasi TensorRT. `SESSION = 2` menjalankan cek NMS cepat dan suite dari arsip sesi 1. `SESSION = 3` menjalankan sensitivitas `best.pt` dan ablasi. Preset `"baselines"` hanya melatih pembanding, `"profil"` hanya menjalankan profil latensi dan L2, dan `"a11a12"` menjalankan profil segmen (A11) dan A12 dalam satu sesi. `SESSION = None` berarti kendali manual lewat `STEPS`. Pemecahan ini diperlukan untuk VisDrone: pada sesi 1 *seed* 42 di T4, pelatihan butuh 6,2 jam, cek

checkpoint + gate + ekspor sekitar 2,7 jam, dan evaluasi TensorRT sekitar 2 jam, sehingga evaluasi PyTorch FP32 terpisah tidak muat dan dilewati (suite sudah memeriksa FP32 pada seluruh data uji).

Melanjutkan ke sesi berikutnya memakai arsip. Bagian 9 (sel 15) menyimpan folder run sebagai `arsip_<model>_seed<seed>.zip` tanpa *engine*, ONNX, dan *checkpoint* periodik. Arsip itu diunggah sebagai dataset Kaggle, `RESUME_FROM` diisi path-nya, dan bagian 3 (sel 5) menyalin isinya ke `/kaggle/working` supaya semua langkah dapat menulis ke folder run. *Engine* TensorRT tidak diarsip karena *engine* terikat pada GPU dan versi TensorRT; karena itu evaluasi TensorRT (bagian 6) membutuhkan ekspor di sesi yang sama. Beberapa `assert` menjaga konsistensi: `RESUME_FROM` tidak boleh diisi bila sesi melatih, dan path pembanding harus terisi sebelum suite.

Fungsi `run_logged()` menjalankan perintah dengan stdout dan stderr ke berkas log, bukan ke keluaran notebook. *Progress bar* Ultralytics membuat keluaran notebook sangat besar, dan itu membuat tahap simpan versi Kaggle sangat lama. Notebook hanya mencetak *epoch* dan mAP50 dari `results.csv` setiap 10 menit, ditambah 25 baris terakhir log. Log dikompres menjadi `.log.gz` bila perintah berhasil. Perintah yang gagal menghentikan notebook (`check=True`), berbeda dengan `!python` bawaan IPython yang mengabaikan kode keluar. Pengecualiannya hanya uji reproduksi, yang dicetak sebagai peringatan.

Dua catatan dari verifikasi sebelumnya relevan di sini. Untuk run berikutnya, folder `/kaggle/working/repro` dan `/kaggle/working/logs` sebaiknya ikut diarsip di bagian 9, karena untuk KITTI log uji reproduksi tidak tersimpan. Selain itu, `BASELINES` untuk KITTI menunjuk ke *checkpoint* lama di Kaggle Models, sedangkan pembanding VisDrone dilatih ulang di T4 dengan bagian 3b. Komentar di bagian 0 menjelaskan alasan pemakaian `last.pt` untuk pembanding: aturannya sama dengan router, sehingga angka pembanding tidak sebanding langsung dengan tabel versi lama yang memakai `best.pt`.

13 Keterbatasan dan Catatan tentang Kode

Bab ini mengumpulkan hal-hal yang perlu diketahui sebelum menyatakan sesuatu tentang kode di sidang. Sebagian adalah batas rancangan yang disengaja, sebagian adalah kekurangan kecil yang ditemukan saat menyusun dokumen ini. Tidak ada yang mengubah angka hasil, tetapi semuanya dapat ditanyakan.

Ketidakpastian latensi tidak masuk ke CI H5. Fraksi campuran f dihitung sekali dari rata-rata latensi dua *build*, lalu diperlakukan sebagai konstanta dalam *bootstrap*. CI H5 karena itu hanya memuat ketidakpastian akurasi, bukan ketidakpastian latensi. Kode menyediakan dua alat untuk menilai bagian yang hilang ini: `h6_per_build.csv`, yang menghitung ulang H6 untuk setiap *build*, dan replikasi A11 dan A12 pada sesi lain. Hasil A11–A12 menunjukkan f dapat bergeser hingga 0,06 antar-sesi. Nilai f juga dipotong ke [0, 1]. Bila router lebih lambat daripada P2-Static, f menjadi 1 dan pembandingnya menjadi P2-Static murni; kegagalan seperti itu tetap terlihat di kriteria H1 latensi.

Batas *timer*. Latensi L1 mencakup kerja host di Python (hitung ambang, peluncuran, sinkronisasi), tetapi tidak mencakup dekode berkas, konversi uint8 ke float, dan NMS. Pilihan ini sama untuk semua varian, sehingga perbandingan antar-varian adil, tetapi angka absolutnya bukan latensi aplikasi utuh. L2 dibuat untuk menjawab keberatan ini, dan statusnya analisis sensitivitas.

Pembanding kaskade dimodelkan, bukan diukur. Latensi kaskade disusun dari komponen yang diukur, dan aktivasinya dipilih dengan top-ρ pada skor uji. Pembanding ini optimistis dan transduktif, sehingga hanya cocok sebagai pembanding tambahan.

Keputusan B2 bergantung pada urutan aliran. Kendali anggaran menjaga aktivasi rata-rata, bukan keputusan per citra. Bila urutan citra berubah, keputusan untuk citra yang sama dapat berubah. Kode

menilai ketahanan aktivasi pada lima urutan acak (`b2_urutan.csv`), tetapi akurasi utama dilaporkan untuk satu urutan, yaitu urutan nama berkas. Selama 20 citra pertama, ambang awal dari val yang dipakai.

FP16 dan FP32 tidak identik. Angka akurasi utama berasal dari *engine* FP16 yang di-*deploy*, dan PyTorch FP32 menjadi pemeriksaan silang. Keputusan rute keduanya berbeda pada 0,1–0,2% citra uji. Verifikasi numerik ketat ($< 10^{-4}$) hanya dilakukan untuk graf ONNX FP32.

Celah NMS cepat dan *letterbox* GPU. NMS cepat identik dengan NMS penuh kecuali bila ada skor yang persis sama di antara kotak yang bertumpuk. Celah ini diukur oleh `check_fast_nms.py` sebelum suite, dan pada suite VisDrone tidak ada *fallback*. *Letterbox* GPU di L2 dapat berbeda satu tingkat piksel dari `cv2`, sehingga kesepakatan keputusan L2–L1 dilaporkan dan dijadikan syarat.

Pembanding KITTI adalah *checkpoint* lama. Untuk VisDrone, Vanilla dan P2-Static dilatih ulang di T4 dengan `build_baseline_trainer()`, sehingga kesetaraan resep dijamin oleh kode ini. Untuk KITTI, notebook memakai *checkpoint* lama dari Kaggle Models. Kesetaraan resep KITTI karena itu bergantung pada catatan run lama, bukan pada kode `router_final`.

Duplikasi kode. Beberapa fungsi punya dua salinan. `pooled_tests()` ada di `cocomix.py` dan di `suite.py` dengan isi yang sama, dan `merge()` memakai salinan di `suite.py`. `cache_split()` dan `fit_gate()` juga punya versi di `gate.py` dan versi lebih luas di `suite.py`. Ini bukan *bug* saat ini, karena kesetaraannya diperiksa (fit ulang gate di suite harus sama dengan `gate.json`), tetapi berisiko bila salah satu diubah di masa depan tanpa yang lain.

Dokumentasi yang tertinggal. README masih menulis bahwa uji reproduksi GPU “belum diuji”, padahal uji itu sudah dijalankan dan lolos (Subbab 9.2). Komentar `# TODO` di `visdrone.yaml` juga sisa penulisan awal. Keduanya tidak memengaruhi eksekusi, tetapi sebaiknya diperbarui sebelum repo dirilis, bersama normalisasi CRLF yang sudah dicatat di verifikasi B4.

Uji kesetaraan yang tidak berdiri sendiri. Uji kesetaraan M0 dan uji *replay* Tahap 4 membutuhkan berkas acuan dari luar folder. Di salinan kode yang bersih, keduanya dilewati, sehingga pihak lain yang hanya punya repo tidak dapat mengulang bukti itu tanpa arsip hasil.

Batas lingkup implementasi. *Engine* dibangun untuk *batch* 1 dan masukan statis 640 × 640. Kode tidak menangani *batch* lebih besar atau resolusi dinamis, dan *engine* tidak portabel antar-GPU atau antar-versi TensorRT. Koreksi `stats[0]` di `coco_eval()` khusus untuk perilaku pycocotools 2.0.x.

14 Pertanyaan Penguji tentang Kode

Bagian ini memuat pertanyaan tentang kode yang mungkin diajukan penguji, beserta jawaban yang merujuk ke bagian kode yang relevan.

Di mana router dalam graf model? Saya tidak melihat modul router di YAML. Memang tidak ada. Graf detektor hanya berisi modul stok Ultralytics, dan keputusan dibuat oleh gate *post-hoc* di luar graf pelatihan (`gate.py`). Saat *deployment*, gate disisipkan ke Stage 1 *engine* sebagai `GateHead` (*pooling* + satu Gemm), dan pemilihan cabang dilakukan operator `If` (`build_if()` di `export.py`). Pemisahan ini disengaja: detektor dilatih tanpa loss gate, sehingga konflik antara loss diskriminatif dan *sparsity* yang ditemukan pada versi pertama tidak mungkin terjadi.

Kalau saat pelatihan kedua cabang selalu dihitung, di mana penghematannya? Penghematan hanya terjadi saat inferensi. Pelatihan sengaja menghitung keduanya agar setiap cabang dapat dipakai sebagai rute mandiri (`DualBranchDetector.loss()` selalu memanggil `forward_dual()`). Saat inferensi, operator `If` di *engine* hanya mengeksekusi subgraf yang dipilih. Rute B butuh sekitar 8,08 GFLOPs, sama dengan Vanilla, dan rute A sekitar 12,61 GFLOPs.

Bagaimana Anda tahu bahwa membuang komponen router lama tidak mengubah detektor? Ada dua bukti di kode. Pertama, `compat.py` memetakan bobot M0 ke graf final, dan `test_m0_equivalence.py` menunjukkan fitur *trunk*, keluaran kedua cabang, dan item loss identik bit per bit dengan model lama. Jadi graf deteksinya sama. Kedua, `test_model.py` memeriksa bahwa tidak ada kunci router di `state_dict` dan semua parameter menerima gradien. Yang berbeda adalah bobot hasil pelatihan ulang, karena resepnya tidak lagi memuat loss router.

Mengapa memakai `last.pt`, bukan `best.pt`? `best.pt` Ultralytics dipilih dari *fitness* cabang A saja, padahal sistem membutuhkan kedua cabang dan target gate adalah selisih A − B. Aturan `last.pt` dikunci di konfigurasi sebelum pelatihan. `check_checkpoint.py` melaporkan selisih `last − best` untuk A, B, dan jarak A − B dengan *bootstrap* berpasangan di val, tanpa mengubah pilihan.

Mengapa gate di-fit di data latih, bukan di val? Target gate membutuhkan label, dan data latih adalah data berlabel terbesar. Val hanya dipakai untuk ambang awal B2 (`thr0`), dan uji tidak pernah dipakai untuk fit. Konsekuensinya, target dihitung pada citra yang sudah dilihat detektor saat pelatihan. Ablasi G5 (biaya label) dan G1 (target) dinilai di val untuk menguji pilihan ini di luar data latih.

Mengapa gate di *engine* FP32, padahal *engine*-nya FP16? Skor gate adalah jumlah 960 suku, dan keputusan bergantung pada perbandingan skor dengan ambang. Pembulatan FP16 pada bobot dan akumulasi dapat membalik keputusan citra yang skornya dekat ambang. Karena itu `Stage1` mengubah fitur ke FP32 sebelum `GateHead`, dan bobot gate disimpan sebagai *buffer* FP32. Biaya tambahannya kecil karena operasinya hanya *pooling* dan satu perkalian vektor.

***Padding* nol pada rute B, apakah tidak menambah deteksi palsu?** Tidak. *Anchor padding* punya skor kelas 0, sedangkan NMS hanya mempertimbangkan kandidat dengan skor di atas conf (0,001 untuk evaluasi, 0,25 untuk *deployment*). `test_nms_route_b_cut` memeriksa bahwa NMS atas 34.000 *anchor* rute B sama dengan NMS atas 8.400 *anchor* aslinya. Biaya *padding* hanya pada salinan ke host, dan itu yang diselidiki profil per segmen dan A12.

Apakah ambang B2 saat *deployment* sama dengan yang dipakai di analisis? Ya, dan ini dijaga di tiga tempat. `SortedWindowQuantile` memakai rumus interpolasi numpy sehingga bit-identik dengan `np.quantile` (`test_budget.py`). Perbandingan selalu dilakukan dalam float32, sama dengan operator `Greater` di *engine*. Saat evaluasi TensorRT, setiap keputusan *engine* dibandingkan dengan perbandingan di host, dan paritas ujung-ke-ujung memeriksa bahwa keputusan pada pengukuran latensi sama dengan pass akurasi.

Bagaimana mAP router dihitung tanpa menjalankan model berkali-kali? COCOeval mencocokkan deteksi dengan GT per citra, sehingga hasil `evaluate()` untuk routing apa pun adalah pilihan per citra dari hasil cabang A atau B. `CocoMix` menyusun pilihan itu dan mereplikasi `accumulate()`. Kesamaan dengan COCOeval langsung diperiksa di setiap run suite (`assert` selisih < 10^{-9}) dan di `test_cocomix.py`.

Mengapa indeks *resample bootstrap* sama untuk semua model? Agar setiap uji berpasangan pada *resample* yang sama, dan agar *pooling* tiga *seed* KITTI sah. Ketiga *seed* berbagi data uji yang sama, dan `boot_seed` serta jumlah citra yang sama menghasilkan indeks yang sama. Rata-rata selisih antar-model dapat dihitung per *resample* (`pooled_tests()`). Untuk VisDrone, *bootstrap* klaster per video ditambahkan karena citra dari video yang sama tidak independen.

Apakah NMS cepat untuk VisDrone mengubah hasil? Tidak, menurut konstruksinya. Hasil NMS cepat hanya dipakai bila syarat yang menjamin kesamaan dengan NMS penuh terpenuhi; selain itu NMS penuh dijalankan (`nms_one()`). Celah satu-satunya (skor kembar) diukur `check_fast_nms.py` di

val dalam FP32 dan FP16 sebelum suite, dan notebook berhenti bila ada satu deteksi pun yang berbeda.

Apa saja yang masuk ke *timer* latensi? Pada latensi ujung-ke-ujung L1: hitung ambang B2 di host, salin citra FP32 dan ambang ke GPU, eksekusi *engine*, salin keluaran mentah, keputusan, dan skor ke host, lalu sinkronisasi. Di luar *timer*: dekode berkas, konversi uint8 ke float, dan NMS. Varian diukur bergantian dengan urutan yang dibalik, pada dua *build engine*, tiga ulangan, dan seluruh aliran uji (`e2e_pass()`).

Bagaimana Anda memastikan Ultralytics yang dipakai adalah versi stok? `require_stock_ultralytics()` memeriksa versi 8.3.240, tidak adanya berkas penanda *fork*, dan sha256 delapan berkas inti terhadap *wheel* PyPI. Fungsi ini dipanggil sebelum pelatihan, di setiap provenans, di suite, dan di uji unit pertama.

Kalau kode ini dijalankan ulang, apakah angkanya sama? Untuk pelatihan, ya, asalkan lingkungannya sama (GPU, *driver*, versi torch/CUDA, *batch*, *worker*); uji reproduksi GPU menunjukkan bobot identik bit per bit (terverifikasi dari log untuk VisDrone, dan tercatat dari keluaran notebook untuk tiga *seed* KITTI). Untuk akurasi dan statistik, ya, karena semua undian acak memakai *seed* tetap dan analisis dapat diulang dari *intermediate* tanpa GPU (`--reuse`). Untuk latensi, tidak persis: latensi bervariasi antar-*build* dan antar-sesi, dan itu sebabnya ada dua *build* per model serta replikasi A11 dan A12.

Mengapa job TensorRT dijalankan di proses terpisah? Dari kodenya, `run_subprocess()` memberi batas waktu dengan menghentikan proses bila macet, menyimpan potongan akhir log, dan memisahkan konteks TensorRT dari memori GPU proses PyTorch tahap A. Komunikasi antarproses hanya lewat berkas (`job.json`, `job_res.json`, `job_dets.pkl`), sehingga job dapat diulang tanpa mengulang tahap A.

Lampiran A. Padanan Label Kriteria

Kode dan berkas hasil memakai penomoran protokol evaluasi (H1–H7). Tesis dan paper memakai penomoran baru (H1–H6), dan kriteria H4 protokol tidak dilaporkan di naskah. Padanannya sama dengan Bahan Tambahan A draf paper.

Tabel A.1. Padanan kriteria tesis dengan label di kode.

Tesis dan paper	Protokol dan kode	Isi	Di mana di kode
H1	H1	Router di antara Vanilla dan P2-Static (mAP50 dan latensi ujung-ke-ujung)	`kriteria.H1_mAP50`, `kriteria.H1_latensi_e2e_efisien`
H2	H2	Router − acak pada aktivasi sama, batas bawah CI 95% > 0	uji H2/H7 | router (B2) − acak (aktivasi sama), `kriteria.H2_router_vs_acak`
H3	H3	ρ parsial skor gate dengan proporsi objek kecil > 0, p Holm < 0,05	`h3_perilaku_skor.csv`, `ringkasan_h3.csv`
—	H4	Perbandingan dengan rancangan tahap pengembangan	tidak dilaporkan
H4	H5	*Overhead* rute ≤ 0,15 ms	`kriteria.H5_rute_A_e2e`, `kriteria.H5_rute_B_e2e`
H5	H6	Router − campuran acak berlatensi sama (kondisi utama)	uji H6 utama | …, `kriteria.H6_utama_status`
H6	H7	|aktivasi uji − ρ| ≤ 0,05 dan H2 lolos	`kriteria.H7_aktivasi`, `kriteria.H7_gain`

Kendali naif tercatat di berkas hasil sebagai `B2` atau `B2 terkunci`, dan kendali efisien sebagai `B2 efisien`.

Lampiran B. Indeks Fungsi dan Kelas Penting

Tabel B.1 adalah indeks cepat untuk menjawab pertanyaan “fungsi mana yang melakukan X”. Kolom terakhir menunjuk subbab dokumen ini.

Tabel B.1. Indeks fungsi dan kelas.

Nama	Berkas	Kegunaan	Subbab
`DualBranchDetector`	`model.py`	Graf dua cabang, *stride* `Detect_A`, `_apply`, `_check_layout`	3.1
`trunk`, `branch_a`, `branch_b`, `forward_dual`	`model.py`	Menjalankan graf per tahap	3.2
`Trunk`, `BranchA`, `BranchB`	`model.py`	Pembungkus tahap untuk ekspor dan evaluasi	3.2
`pool4`	`model.py`	Fitur gate 960 dimensi [Pers. (2)]	3.3
`load_detector`, `reset_anchor_cache`	`model.py`	Memuat *checkpoint*; membuang *cache anchor*	3.4
`BranchLoss`, `DualBranchLoss`	`loss.py`	Loss stok per kepala; L_A + 0,7 · L_B [Pers. (1)]	3.5
`DualBranchTrainer`, `BranchBValidator`	`train.py`	Trainer dan validasi cabang B	3.6
`build_trainer`, `build_baseline_trainer`	`train.py`	Pelatihan detektor dan pembanding	3.6
`make_loader`	`data.py`	`DataLoader` biasa, *batch* 1, urutan tetap	4.1
`small_area_of`, `_scale_xyxy`	`data.py`	Batas objek kecil dan koordinat asli	4.2
`nms_one`, `to_dets`, `jdict`	`data.py`	NMS (dan NMS cepat), format COCO	4.3
`build_coco_gt`, `coco_eval`	`data.py`	GT resolusi asli; COCOeval dengan koreksi `stats[0]`	4.4
`per_image_soft_small`	`data.py`	S_k per citra [Pers. (3)]	4.5
`cache_split`, `soft_small_delta`	`gate.py`	*Cache* fitur dan deteksi; Δ soft_small	5.1–5.2
`fit_gate`, `save_gate`, `load_gate`, `score_np`, `GateHead`	`gate.py`	Fit, lipatan [Pers. (4)], simpan, skor	5.2–5.3
`b2_thresholds`, `b2_decide`, `thr0_from_val`, `B2Controller`	`budget.py`	Kendali anggaran [Pers. (5)]	5.4
`SortedWindowQuantile`, `b2_host_overhead`	`budget.py`	Kendali efisien; biaya host	5.5
`Stage1`, `IOCast`, `export_a3`	`export.py`	Modul ekspor dan presisi campuran	6.2
`build_if`, `_prefixed_graph`	`export.py`	Graf ONNX `If`	6.3
`verify_a3`	`export.py`	ONNX Runtime vs PyTorch	6.4
`build_engine`, `A3Engine`	`trt.py`	Membangun dan menjalankan *engine*	6.5
`run_split`, `metrics`	`evaluate.py`	Evaluasi satu split	6.6
`CocoBootstrap`, `CocoMix`, `Router`	`cocomix.py`	Metrik routing dari `evalImgs`; target per citra	7.1
`UnionBoot`, `FastUnionBoot`, `boot_engine`	`cocomix.py`	*Bootstrap* berpasangan cepat	7.2
`boot_tests`, `_pval`, `holm`, `partial_spearman_p`, `pooled_tests`, `cluster_index`	`cocomix.py`	Uji, nilai p, Holm, ρ parsial, *pooling*, klaster	7.3
`top_rho`, `random_masks`	`cocomix.py`	Pemilihan top-ρ dan *mask* acak	7.1, 7.7
`load_static`, `static_with_feats`, `P3Density`, `S1Heur`	`baselines.py`	Pembanding dan heuristik	7.4
`stage_a`, `export_suite`, `verify_suite`	`suite.py`	Tahap A suite	7.5
`job_main`, `acc_pass`, `lat_pass`, `e2e_pass`	`suite.py`	Job TensorRT	7.6
`analyze`	`suite.py`	*Bootstrap* dan kriteria	7.7
`run_model`, `zip_model`, `merge`, `pareto_plot`	`suite.py`	Orkestrasi dan gabungan	7.8
`Split`, `run_ablation`, `boot_ablation`, `PAIRS`	`ablation.py`	Ablasi	8.1
`best_epoch_from_results`, `evaluate_pair`, `verdict`	`ckcheck.py`	Cek *checkpoint*	8.2
`Timer`, `profile_pass`, `derive`	`profile.py`	Profil per segmen	8.3

Nama	Berkas	Kegunaan	Subbab
`GpuLetterbox`, `nms_gpu`, `make_l2_ops`, `l2_pass`	`latency_l2.py`	Latensi L2	8.4
`transpose_output`, `a3t_two_stage`, `parity`	`a12.py`	A12	8.5
`require_stock_ultralytics`, `provenance`, `load_config`	`env.py`	Penjaga lingkungan dan provenans	9.1
`state_hash`, `max_abs_diff`	`repro.py`	Sidik jari bobot	9.2
`map_m0_state_dict`	`compat.py`	Pemetaan bobot M0	9.3